

Agentic AI Interview Preparation Guide

From LLM apps to autonomous, tool-using, multi-agent systems — architecture, LangChain & LangGraph, Agentic RAG, security, production operations, and a full interview playbook.

Built for: Data Scientist • ML Engineer • AI Engineer • Generative AI Engineer

RAG Engineer • LLM Engineer • LLM Solution Architect

Agentic AI Engineer • LangChain Engineer • LangGraph Engineer

53

PAGES

32

CORE TOPICS

18

AGENT TYPES

16

DESIGN PATTERNS

5

CAPSTONE PROJECTS

75+

INTERVIEW Q&A

50+

SCENARIO DRILLS

"An agent isn't defined by having an LLM in the loop — it's defined by the LLM deciding what happens next."

Every page pairs its topic with a **Scenario Prep** drill — a real situational question plus the approach to answer it — so this reads as interview rehearsal, not just reference material.

Deepika Chintala

Senior AI/ML Engineer — Generative AI & Agentic Systems

[linkedin.com/in/deepikachintala](https://www.linkedin.com/in/deepikachintala)

What This Guide Covers

This is a working reference, not a glossary. Every topic is built the way I'd actually explain it to another engineer on my team — what it is, why it exists, how it's built, when to reach for it, and when to leave it alone — followed by the interview questions it tends to produce.

How it's organized

FOUNDATIONS (P.4–16)

What Agentic AI actually is, core characteristics, six architecture tiers, the agent execution loop, agents vs. workflows, the 18 agent archetypes, and 16 design patterns.

PATTERNS & BUILD (P.17–27)

The Python ecosystem, LangChain and LangGraph from an agentic lens, memory, Agentic RAG, multi-agent orchestration, and tool calling — each with runnable code.

PRODUCTION (P.28–36)

Security, a production reference architecture, evaluation, a 25-issue troubleshooting playbook, observability, and cost optimization — the part most study guides skip.

PROJECTS & INTERVIEW PREP (P.37–53)

Three fully coded projects, five company-inspired capstone architectures, a tiered interview question bank, scenario drills, cheat sheets, and a 7-day revision plan.

A note on the capstones

The Meta, Apple, Tesla, and Google-inspired projects are **interview-style capstone designs** built around each company's public product domain — not real internal architectures. Nothing here is claimed as what those companies actually run in production. The banking project reflects patterns genuinely common in financial-services engineering, including why full autonomy is deliberately withheld from high-risk actions.

How to use it before an interview

Read foundations start to finish once. After that, use the cheat sheets (p.49–50) and the 50 must-remember points (p.52) as your day-before review, and the 7-day plan (p.53) if you're starting from zero.

Tip: Every code sample uses current, non-deprecated library APIs as of late 2025/2026 tooling. If a library ships breaking changes after this was written, check its migration guide before trusting the exact method names — the underlying architecture reasoning still holds.

SCENARIO PREP

Scenario: You have one week before an interview and haven't touched LangGraph yet. What do you prioritize?

Approach: Skip the framework survey. Build Project C (p.37–39) end to end; depth on one working example beats breadth across five you've only read about.

WHAT "SCENARIO PREP" ADDS BEYOND THE REGULAR Q&A

The standard Q&A blocks throughout this guide test whether you know a concept. The green Scenario Prep box on every page tests something different: what you'd actually say or do when a real interview moment puts you on the spot — a follow-up you didn't prepare for, a live whiteboard challenge, a detail changed mid-answer. Read both, but rehearse the scenario boxes out loud; that's the part that transfers to the actual room.

Table of Contents

01	Agentic AI Fundamentals	4	17	Agent Evaluation	32
02	Core Characteristics	6	18	Troubleshooting Playbook	33
03	Agentic AI Architecture (6 tiers)	7	19	Observability	35
04	Agent Execution Loop	10	20	Cost Optimization	36
05	Agents vs. Workflows	12	21	Executable Python Projects	37
06	18 Types of AI Agents	13	22	Five Enterprise Capstones	40
07	16 Agentic Design Patterns	15	23	Capstone Interview Prep	43
08	Python Ecosystem	17	24	Interview Question Bank	44
09	LangChain for Agentic AI	19	25	Scenario-Based Questions	46
10	LangGraph for Agentic AI	21	26	ML Engineering + Agentic AI	47
11	Agent Memory	23	27	The LLM Solution Architect Lens	48
12	Agentic RAG	24	28	When NOT to Use Agentic AI	48
13	Multi-Agent Systems	26	29	Cheat Sheets	49
14	Tool Calling	27	30	How to Explain It in an Interview	51
15	Agent Security	28	31	Top 50 Must-Remember Points	52
16	Production Architecture	30	32	7-Day Interview Revision Plan	53

How this maps to your target roles: Sections 1–7 and 26–27 are architecture/design-thinking heavy (Solution Architect, senior AI Engineer). Sections 8–14, 21 are hands-on build sections (RAG/LLM/LangChain/LangGraph Engineer). Sections 15–20 are what separates a mid-level agent builder from a senior one in interviews.

HOW TO READ THE PAGE MARKERS

Q. / A.

Interview-style question and a senior-level spoken answer, not a textbook definition.

TIP / CALLOUT BOXES

The single sentence I'd actually say out loud if asked to summarize the page.

P0 P1 P2

Priority tags on page 52 — P0 must be automatic recall, P1 solid, P2 a bonus if it comes up.

Every code block in this guide is meant to be typed or copied and actually run — not just read. The three executable projects on pages 37–39 are the fastest way to turn this from memorized material into demonstrable experience before an interview.

SCENARIO PREP

Scenario: An interviewer says 'walk me through whatever part of Agentic AI you know best.' Where do you start?

Approach: Start from whichever section you know best, not page 1. Confident depth on one topic lands better than a nervous tour through all 32.

What Is Agentic AI?

Every one of these terms gets used interchangeably in job descriptions, and mixing them up in an interview is an easy way to look junior. They sit on a spectrum of how much a system decides for itself:



TERM	WHAT DECIDES THE NEXT STEP	REAL EXAMPLE
Traditional software	The developer, entirely. Every branch is a hand-written if/else.	A fee calculator in a banking app
Automation / RPA	A fixed script triggered by an event. No reasoning, just replay.	Nightly batch job that reconciles two ledgers
Chatbot	Intent matching against a decision tree.	"Press 1 for balance" style IVR bot
LLM application	A model generates text for one prompt, no tools, no memory of state beyond the chat turn.	A single-shot "summarize this email" feature
RAG application	The model still just generates, but the prompt is grounded with retrieved context.	"Ask our policy PDFs a question" search bar
AI Agent	The <i>model itself</i> decides which tool to call and when to stop, in a loop.	An agent that checks a customer's account, then decides whether to open a ticket
Agentic AI	The agent plans multi-step goals, reflects on its own output, and adapts the plan mid-task.	An agent that investigates a suspicious transaction, gathers evidence, revises its theory, and drafts a report
Multi-Agent System	Several specialized agents collaborate, each owning part of the goal, coordinated by a supervisor.	Fraud agent + policy agent + risk agent working one case together

The one-line definition interviewers want: Agentic AI is an LLM-driven system that can perceive a goal, plan a sequence of actions, call tools to act on the world, observe the results, and revise its plan — with the reasoning loop, not a human, deciding what happens next.

KEY POINTS TO REMEMBER

- Not every LLM app is an agent. If there's no loop and no tool-driven decision, it's just a generation step.
- RAG is a capability an agent can use, not a synonym for agent.
- "Agentic" describes a degree of autonomy, not a specific framework.
- A single agent with tools is still "AI Agent"; multi-step planning + self-correction is what earns "Agentic AI."

REAL-WORLD EXAMPLE

A mid-size bank I'd model this on had a "document Q&A chatbot" that everyone called an agent. It took a question, retrieved from a vector store, generated an answer — classic RAG, zero decision-making. When they later added the ability for it to decide whether to also query the core banking API for account-specific numbers, check whether the retrieved policy was still current, and re-retrieve if the first pass looked thin, that's the point it became an actual agent. Same UI, completely different engineering problem underneath.

Common interview trap: Being asked "is a chatbot an agent?" and answering yes or no. The right answer is "it depends whether the model is deciding the next action or just generating the next reply" — then give the test above.

SCENARIO PREP

Scenario: Interviewer: 'Is our current chatbot an agent?' You've never seen their system.

Approach: Before answering, ask what decides the next line of conversation: a decision tree, or the model. One clarifying question beats guessing at an architecture you can't see.

Fundamentals — Interview Q&A

Q. What's the actual difference between an AI Agent and Agentic AI?

A. An AI Agent is the unit: one LLM, a set of tools, and a loop that lets it decide which tool to call. "Agentic AI" describes systems built from one or more agents that exhibit planning, tool use, memory, and self-correction as first-class behaviors, not just a single decision. In practice, people use "agentic" as the umbrella and "agent" as the building block.

Q. Why wouldn't you just use RAG instead of building a full agent?

A. Because RAG answers "what does the document say" questions well but can't take action, chain multiple retrievals, or decide it needs a different data source mid-answer. If the task is genuinely "look up and answer," RAG alone is cheaper, faster, and more predictable. You reach for an agent when the task requires deciding *what to do next* based on intermediate results, not just what to say.

Q. Give a banking example where a workflow is enough and an agent would be overkill.

A. Calculating a loan's monthly payment from principal, rate, and term is pure math with no ambiguity. It's a deterministic function, not a model call. I've seen teams wrap this in an "agent" for demo appeal and it just adds latency, cost, and a new failure mode for a problem that had zero decision points to begin with.

Q. Scenario: A stakeholder says "let's make our support chatbot agentic." What do you ask before agreeing?

A. What decisions does the bot currently escalate to a human, and could it make those decisions safely with tool access? If today's bot already answers from a fixed FAQ, "agentic" usually means: give it tools to check order status, issue refunds under a threshold, and re-route to policy documents when the FAQ doesn't cover it, with clear boundaries on what it's allowed to execute without approval. I'd scope the tool list and the approval boundary before touching any code.

Q. What's a common interview trap around "autonomous agents"?

A. Assuming more autonomy is always better. A fully autonomous agent that can execute irreversible actions (wire a payment, delete a record, send an external email) without a human checkpoint is a liability, not a feature, in most enterprise contexts. Senior candidates bring this up unprompted.

REAL-WORLD FRAMING I USE

When I explain this to a non-technical stakeholder, I skip the taxonomy and use one test: *"If I changed the input slightly, would the system take a genuinely different sequence of actions, or just produce different words?"* Traditional software and simple RAG apps almost always do the latter. That test alone filters out most "we want an agent" requests that are really just "we want better prompts."

Interview tip: Don't lead with definitions memorized from a blog post. Lead with the decision test above, then map it to the term. It signals you understand the distinction rather than having recited it.

SCENARIO PREP

Scenario: You're asked to define Agentic AI in one sentence with zero prep time.

Approach: Fall back to the loop test: 'the model decides the next action, not just the next words.' It's short, it's defensible, and it gives you an opening to add examples.

Core Characteristics of Agentic AI

These six traits are what an interviewer is actually probing for when they ask "what makes something agentic." A system needs most of these, not just one, to earn the label.

AUTONOMY

Executes steps without a human approving each one. Bounded by tool permissions, not by the model's judgment alone. *Position: sits at the orchestrator, gating which actions run unattended.*

GOAL-ORIENTATION

Works toward an outcome, not a single response. "Investigate this transaction" is a goal; "summarize this text" is a task. *Drives how many loop iterations the agent is allowed.*

PLANNING

Breaks a goal into an ordered (or re-orderable) set of steps before or during execution. *Lives between the LLM and the tool-selection layer.*

TOOL USE

Calls external functions, APIs, or retrievers and incorporates real results, not just generated text. *The boundary between "language model" and "agent."*

MEMORY

Retains state across steps and, often, across sessions — what's been tried, what worked, what the user already said. *Backs the planner so it doesn't repeat failed actions.*

ADAPTABILITY

Revises the plan when an observation contradicts the original assumption — a tool fails, data looks wrong, a policy blocks an action. *What separates a script from an agent under real conditions.*

MINIMAL PYTHON SHAPE (ILLUSTRATES WHERE EACH TRAIT LIVES)

```
# A trimmed agent loop – each comment marks a characteristic above
def run_agent(goal: str, tools: dict, llm, memory) -> str:
    plan = llm.plan(goal, memory.recall(goal))          # planning + memory
    for step in range(MAX_STEPS):                        # autonomy, bounded
        action = llm.decide_next_action(plan, memory.trace)
        if action.name == "final_answer":
            return action.output                          # goal-orientation: stop condition
        result = tools[action.name](**action.args)       # tool use
        memory.trace.append((action, result))
        if llm.detects_contradiction(result, plan):      # adaptability
            plan = llm.replan(goal, memory.trace)
    return "max steps exceeded – escalate to human"
```

INTERVIEW Q&A

Q. Which of these six traits is most often faked in a demo?

A. Adaptability. Most demo agents follow a fixed plan and just happen to succeed because the demo path was tested. Ask what happens when a tool call returns an error or an unexpected schema; that's where real adaptability shows itself, or doesn't.

Q. Can an agent have memory but not be "agentic"?

A. Yes. A stateful chatbot that just appends conversation history is remembering, not reasoning about that history to change its strategy. Memory is necessary but not sufficient.

Tip: If asked to design an agent, name which of these six traits the use case actually needs before writing any architecture — most production agents deliberately skip one or two (often full autonomy) for safety.

SCENARIO PREP

Scenario: A panel challenges 'why does this agent need memory?' the moment you mention it.

Approach: Don't defend memory reflexively. State what actually breaks without it, and if the task doesn't need it, say so.

The Enterprise Agentic AI Stack

This is the full layer stack a production agent sits inside. Not every project needs every layer — the six architecture tiers on the next two pages show which layers turn on as complexity grows.



ENTRY User → App/API → Auth	REASONING Agent Orchestrator → LLM → Planner	KNOWLEDGE & ACTION Memory → Tools → RAG → Vector DB → External APIs → DBs	CLOSING THE LOOP Execution → Observation → Evaluation → Feedback → Response
---------------------------------------	--	---	---

LAYER	ROLE
Application / API	Auth, rate limiting, request shaping. The only layer that talks to the outside world directly.
Agent Orchestrator	Owns the loop: decides whether to call the LLM again, a tool, or stop. In LangGraph this is the graph runtime.
LLM	Reasoning engine. Produces the next action given goal + trace, not just a chat reply.
Planner	Decomposes the goal into steps; re-invoked when the evaluator flags a bad plan.
Memory	Short-term (this run) and long-term (across runs) state store.
Tools / RAG / Vector DB / External APIs / DBs	Everything the agent can act on or retrieve from. Each is a bounded, permissioned capability, not open access.
Execution	Actually invokes the chosen tool with validated arguments.
Observation → Evaluation → Feedback	Result is checked against expectation; feeds either the next planning step or a logged metric.

Recurring design question: before adding any layer, ask — can this be deterministic? Do we need retrieval at all? Does one agent solve it, or do we genuinely need a planner and a critic? Every layer you add is a new failure mode, a new cost, and a new thing to monitor.

WHO OWNS WHAT, IN PRACTICE

PLATFORM / MLOPS TEAM API gateway, orchestrator runtime, observability stack, vector DB infra, cost dashboards — the parts that are the same regardless of which agent runs on top.	AI/ML ENGINEER (THIS GUIDE'S SCOPE) Planner logic, tool definitions and schemas, retrieval strategy, memory design, evaluation criteria, prompt and model selection — the parts specific to what the agent actually does.
---	---

Interview tip: When asked to "design an agent architecture," sketch this exact eight-box flow first, then say which boxes your specific use case needs. Interviewers reward candidates who scope down before scoping up.

SCENARIO PREP
Scenario: You get 5 minutes to whiteboard 'an AI system' with no other spec.
Approach: Draw the 8-layer stack, then ask which layers deserve depth. Trying to detail all eight in five minutes just means rushing the one that actually matters.

Six Architecture Tiers (1–3)

Same underlying stack, different subsets turned on. Pick the lowest tier that solves the problem — each tier up adds real operational cost.

	TIER 1: SIMPLE AGENT	TIER 2: TOOL-USING AGENT	TIER 3: AGENT + RAG
Components	LLM + loop, no external tools	+ 1–5 bounded tools (calculator, API, DB)	+ retriever + vector DB
Data flow	Prompt in → reasoning → answer out	Prompt → decide tool → call → incorporate result → answer	Prompt → decide if retrieval needed → retrieve → ground → answer
Decision flow	Single reasoning pass	Tool selection per step, may loop 2–4x	Retrieval is itself a decision, not automatic
Failure mode	Hallucinated fact, no way to check	Wrong tool chosen; malformed arguments	Retrieves irrelevant chunks; answers off stale docs
Security	Prompt injection only	+ tool-argument injection, needs input validation	+ document poisoning, metadata filtering
Monitoring	Token count, latency	+ tool call success rate	+ retrieval precision/recall
Scaling	Trivially horizontal	Tool rate limits become the bottleneck	Vector DB read throughput becomes the bottleneck
Typical cost driver	Output tokens	Tool + LLM round trips	Embedding + retrieval + larger context

Interview framing: "Tier 1 is basically an LLM app with a stop condition; the real jump in engineering complexity happens at Tier 2, the moment a tool call can fail or return something unexpected and the agent has to handle it."

COMMON QUESTIONS ON THESE THREE TIERS

- Q. When is Tier 1 (no tools) actually the right production choice?**

A. When the task is pure language transformation with no external facts needed: rewriting tone, classifying intent, extracting structured fields from text already in the prompt. Adding tools there is pure overhead.
- Q. What's the single biggest jump in engineering effort between these tiers?**

A. Tier 2. Once a tool call can fail, return a different schema than expected, or take five seconds instead of 200ms, you need retries, timeouts, and argument validation, none of which Tier 1 ever required.
- Q. Why does Tier 3 list "decides if retrieval needed" as a step, instead of always retrieving?**

A. Always-retrieve wastes a round trip on questions the model can already answer confidently, and worse, it can pull in irrelevant context that drags the answer off track. A real Agentic RAG system treats retrieval as an action the agent chooses, not a fixed pipeline stage. That's covered in full on page 22.

REAL-WORLD EXAMPLE

A Tier 2 tool-using agent I'd design for transaction lookups gets maybe four tools: `get_account_balance`, `get_transaction_history`, `flag_for_review`, and `send_notification`. That's it. Every extra tool added "just in case" measurably increases how often the model picks the wrong one — tool sprawl is a real production failure mode, not a theoretical one.

SCENARIO PREP

Scenario: Interviewer pushes: 'why not always use the most capable architecture tier?'

Approach: Cite the operational cost table directly: Tier 2 already adds retries, timeouts, and validation that Tier 1 never needed. Over-architecting has a real price tag.

Six Architecture Tiers (4–6)

	TIER 4: AGENT + RAG + MEMORY	TIER 5: PRODUCTION ENTERPRISE AGENT	TIER 6: MULTI-AGENT SYSTEM
Components	+ short/long-term memory store	+ gateway, cache, queue, full observability, evaluation harness	+ supervisor + 3–6 specialist agents
Data flow	Retrieval augmented by what's been discussed/tried before	Request → gateway → async orchestrator → agent → response, with caching at each layer	Supervisor decomposes → delegates to specialists → aggregates
Decision flow	Plan considers memory conflicts (contradicts earlier answer?)	Circuit breakers decide fallback vs retry vs escalate	Supervisor decides which specialist(s), sequential or parallel
Failure mode	Stale/incorrect memory biases new answers	Cascading failure if one downstream service degrades	Agent deadlock, duplicate work, conflicting outputs
Security	+ memory poisoning, cross-user data leakage	+ full RBAC, secrets management, audit logging	+ inter-agent trust boundaries, message validation
Monitoring	+ memory hit rate, staleness checks	Full distributed tracing, cost per request, SLA dashboards	+ per-agent trajectory, handoff latency
Scaling	Memory store read/write volume	Horizontal at every layer; the LLM gateway is usually the real limit	Coordination overhead grows faster than agent count
Typical cost driver	Memory summarization LLM calls	Aggregate of every layer, controlled via caching + routing	N agents × LLM calls; easy to 4–6x cost silently

INTERVIEW Q&A

Q. How do you decide when a project justifies Tier 6 (multi-agent) over a single well-designed Tier 5 agent?

A. I look for genuinely separable sub-domains with different tool sets and different failure tolerances. Fraud detection and policy lookup are different enough in a bank that one agent juggling both tends to confuse its own tool selection. If one agent with a clean prompt and a focused toolset can do it, I stay at Tier 5; multi-agent adds coordination cost that has to be justified by a real separation of concerns, not just "it sounds more sophisticated."

Q. What's the most underestimated cost when moving from Tier 3 to Tier 5?

A. Observability engineering, not compute. Teams budget for the LLM calls and forget that a production agent needs full request tracing (which tool, which arguments, which retrieval, in what order) just to debug a single wrong answer, and that tooling is often a bigger build than the agent logic itself.

KEY POINTS TO REMEMBER

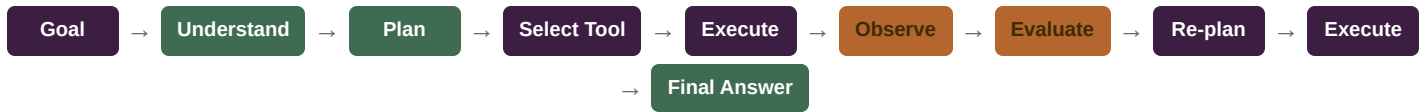
- Tier jumps are about new failure modes, not new features — every "+" in the table above is also a "+1 thing that can break."
- Multi-agent cost scales faster than agent count because of coordination overhead, not just $N \times \text{LLM calls}$.
- Memory is the tier where privacy and data-leakage risk enters the picture for the first time.
- Tier 5's caching and routing layers usually save more money than any single model-selection decision.

SCENARIO PREP

Scenario: You're asked to justify a Tier 6 (multi-agent) choice you personally wouldn't have made.

Approach: If it fails the separable-sub-domain test on p.26, say so. 'I'd start at Tier 5 unless X' lands better than defending a tier you'd never have chosen.

The Agent Execution Loop



Every agent framework implements some version of this loop. The named patterns below are really just different disciplines for running it.

PATTERN	WHAT IT ACTUALLY DOES	WHEN TO REACH FOR IT
ReAct	Interleaves reasoning text ("Thought: ...") with actions ("Action: tool_x") in a single running transcript — the model narrates its own reasoning before each tool call.	Default choice for most tool-using agents; easy to debug because the reasoning is visible in the trace.
Plan-and-Execute	Produces the full step list up front, then executes each step, only revisiting the plan if something fails.	Tasks where the steps are mostly predictable and you want fewer, cheaper LLM calls than ReAct's per-step reasoning.
Reflection	After producing an answer, the model (or a second call) critiques its own output against the goal before returning it.	High-stakes outputs where a second pass catches errors cheaply — drafting policy-sensitive text, code review.
Self-correction	Reflection's action-oriented sibling: when an evaluation step fails, the agent retries with a revised approach instead of just flagging the issue.	Anywhere a first attempt is expected to sometimes miss — SQL generation, data extraction.
Tool / Function calling	The model outputs a structured call (name + JSON arguments) that the runtime executes and feeds back as an observation.	The mechanism underneath nearly every pattern above — not a competing choice, a shared building block.
Iterative reasoning	General term for looping reasoning → action → observation until a stop condition, regardless of which pattern drives it.	Umbrella term; use it when describing the loop generically in interviews.

Interview framing: ReAct and Plan-and-Execute are not mutually exclusive — a lot of production agents plan at a coarse level, then use ReAct-style reasoning within each planned step. Saying this in an interview signals real implementation experience, not just terminology recall.

SCENARIO PREP

Scenario: Asked to compare ReAct vs. Plan-and-Execute for a project you haven't actually built.

Approach: Reason from cost and predictability instead of memorized definitions: 'ReAct costs more per step but adapts mid-task; I'd pick Plan-and-Execute if the steps are mostly known upfront.'

Worked Example: The Loop End-to-End

USER REQUEST

"Find the best cloud architecture for processing 1 TB of banking transaction data."

#	STEP	WHAT THE AGENT DOES
1	Understand	Parses the ask into sub-goals: volume (1TB), domain (banking → compliance matters), and deliverable (a recommendation, not raw data).
2	Break task	Plans three sub-tasks: research applicable architectures, check current internal constraints, compare options against cost/latency/compliance.
3	Select tools	Chooses a documentation-search tool and an internal-database query tool over, say, a code execution tool — nothing here requires running code.
4	Search docs	Retrieves current AWS/GCP/Azure batch and streaming reference architectures for large-scale structured data.
5	Query database	Pulls the bank's existing data volume trends and current cloud spend to ground the recommendation in reality, not generic advice.
6	Analyze results	Cross-references: is 1TB a one-time load or daily? That single fact changes the answer from "batch ETL" to "streaming pipeline."
7	Compare architectures	Weights a Glue/EMR-style batch approach against a Kafka + streaming-warehouse approach on cost, latency, and operational complexity.
8	Re-plan	Realizes the original plan didn't account for compliance data-residency rules and adds a step to check them before finalizing.
9	Final recommendation	Produces a scoped answer with the assumption stated explicitly ("assuming daily batch, not real-time") rather than a generic architecture essay.

INTERVIEW Q&A

- Q. Why does step 8 matter more than it looks like it should?

A. Because it's the only step that demonstrates adaptability. Anyone can chain steps 1–7 as a fixed pipeline; re-planning mid-task in response to a gap the agent itself noticed is what separates an agent from a scripted workflow with an LLM bolted on.
- Q. How would you cap this loop so it doesn't run forever on an ambiguous request?

A. A hard step limit (e.g., 8–12 iterations) plus a confidence check: if the evaluator can't produce an answer above a threshold after N steps, the agent returns its best partial answer with what's still uncertain, rather than looping indefinitely or guessing.

Tip: When asked to trace an agent's reasoning in an interview, narrate it exactly like this table — step, action, why — instead of a paragraph. It mirrors how the actual trace looks in LangSmith or Langfuse and shows you've debugged real agents, not just built demos.

SCENARIO PREP

Scenario:

Interviewer changes one detail of the worked example mid-answer ('what if it's a one-time load, not daily').

Approach:

Trace which step actually changes. Here it flips the recommendation at step 6, and walking through why shows you understand the dependency instead of the memorized sequence.

Agents vs. Workflows vs. Agentic Workflows

This is one of the most common senior-level interview topics because it tests judgment, not recall.

WORKFLOW
Fixed, predefined steps. The developer encodes every branch. Predictable and cheap, but brittle to anything unforeseen.

AGENT
Dynamic decision-making. The model chooses the next action based on the current state, not a hard-coded branch.

AGENTIC WORKFLOW
A deterministic skeleton (fixed stages) with an intelligent agent making the decisions *within* a stage. Most production systems live here.

FEATURE	WORKFLOW	AGENT	AGENTIC WORKFLOW
Control flow	Fixed	Dynamic	Hybrid
Reasoning	Limited	High	High, scoped
Tool selection	Predefined	Dynamic	Mixed
Predictability	High	Lower	Medium–High
Cost	Lower	Higher	Optimized
Reliability	High	Variable	High

WHEN SHOULD I USE A WORKFLOW INSTEAD OF AN AGENT?

Use a workflow when the steps and their order are known in advance and don't change based on content — onboarding a new user, running a nightly reconciliation job, sending a scheduled report. Use an agent when the *right next step depends on what the previous step returned* in a way you can't fully enumerate ahead of time. Most real systems aren't purely one or the other: a loan-processing pipeline might be a fixed workflow (submit → verify → score → decide) where the "verify" and "score" stages each contain an agent that decides which documents to check or which data sources to query.

INTERVIEW Q&A

- Q. Your team wants to rebuild an existing rules-based workflow as "an agent." What do you ask?**

A. What's actually failing today: is the rules engine producing wrong outcomes, or is it just hard to maintain? If the rules are correct but brittle to change, that's a maintainability problem, and an agent doesn't fix maintainability, it trades determinism for flexibility you may not need. I'd only agree if the failure mode is genuinely "the input space is too large to enumerate as rules."
- Q. Can an agentic workflow have a lower failure rate than a pure agent, even though it's less flexible?**

A. Yes, and this surprises people. Constraining the agent to a known set of stages means the model only has to make a good decision within a bounded context each time, instead of deciding the entire trajectory freely, and that bounding reduces the surface area for cascading errors.

Key point: "Agentic workflow" is the answer to give when asked how you'd actually ship something to production — it's the pragmatic middle interviewers are usually fishing for.

SCENARIO PREP
Scenario: You're asked to defend rebuilding a legacy rules engine as an agent.
Approach: Ask what's actually failing first. That question alone often reveals 'agent' is being used as a buzzword, and asking it signals judgment over compliance.

18 Types of AI Agents (1–9)

TYPE	CORE IDEA	REAL-WORLD EXAMPLE
1. Simple Agent	Single LLM call with a stop condition, no tools. The floor of the spectrum.	Text classifier that labels a support ticket's category
2. Reactive Agent	Responds to the immediate input only — no planning, no memory of past turns.	Rule-triggered alert bot: "balance below \$0 → notify"
3. Tool-Using Agent	Calls external functions/APIs and incorporates real results into its answer.	Agent that checks live account balance before answering
4. Planning Agent	Decomposes a goal into an ordered set of sub-tasks before acting.	Agent that breaks "prepare a credit report" into fetch → verify → summarize
5. ReAct Agent	Interleaves visible reasoning with each action, one step at a time.	Debuggable customer-support agent where every tool call is preceded by a logged "Thought"
6. Reflection Agent	Critiques its own draft output against the goal before finalizing it.	Contract-summary agent that re-checks its summary against the source clause by clause
7. Autonomous Agent	Executes multi-step tasks with minimal human checkpoints, within defined tool permissions.	Agent that fully handles password-reset tickets end to end
8. Memory-based Agent	Persists and recalls state across sessions, not just within one run.	Personal finance assistant that remembers stated budget goals across visits
9. RAG Agent	Decides when and what to retrieve from a knowledge base as part of its reasoning.	Internal policy assistant that pulls the current version of a compliance doc

Interview tip: These aren't mutually exclusive categories — a production agent is usually 4–5 of these at once (e.g., a Tool-Using + Memory-based + ReAct agent). If asked "what type of agent is this," describe the combination rather than forcing a single label.

KEY ADVANTAGES / DISADVANTAGES SNAPSHOT

WHERE THESE SHINE

- Reactive & Simple: near-zero latency, fully predictable cost
- Planning & ReAct: transparent, debuggable multi-step reasoning
- Memory-based & RAG: grounded, personalized, current answers

WHERE THESE STRUGGLE

- Reactive: can't handle anything requiring context from earlier
- Autonomous: risk of irreversible actions without a checkpoint
- Memory-based: staleness and cross-session data leakage risk

SCENARIO QUESTION

Q. You're building an internal HR assistant that answers policy questions and can also submit PTO requests. Which of the nine types above apply?

A. At minimum: RAG Agent (policy lookup), Tool-Using Agent (submitting the PTO request itself), and Memory-based Agent (remembering the employee's role/location so answers reflect the right policy variant). If PTO submission should require confirmation, add Human-in-the-loop from the next page. That's the kind of layering interviewers want to see you reason through live.

SCENARIO PREP

Scenario: Asked to classify an unfamiliar system into one of the 18 agent types.

Approach: It's rarely just one type. Name the two or three that combine, such as a Tool-Using plus Memory-based agent; that's closer to how real systems get described.

18 Types of AI Agents (10–18)

TYPE	CORE IDEA	REAL-WORLD EXAMPLE
10. Coding Agent	Writes, runs, and iterates on code using an execution tool and test feedback.	Agent that fixes a failing unit test by reading the stack trace and patching the function
11. Research Agent	Multi-source information gathering with synthesis, often iterative.	Agent that compiles a competitor-pricing summary from several web sources
12. Multi-Agent System	Several specialized agents collaborate under coordination.	Fraud + policy + risk agents jointly investigating one case
13. Supervisor Agent	Owens task delegation and result aggregation across specialist agents; doesn't do the work itself.	Routes "check this loan application" to underwriting, compliance, and credit-check agents
14. Router Agent	Classifies the request and forwards it to the right specialist or workflow.	Support-ticket agent that decides "billing" vs "technical" vs "cancellation" queue
15. Critic Agent	Evaluates another agent's output against criteria without producing the original answer itself.	Reviews a generated legal summary for omitted clauses before it reaches a human
16. Planner Agent	Specialized purely in decomposing goals into executable steps for other agents to run.	Breaks "migrate this pipeline to Azure" into a task list for an executor agent
17. Executor Agent	Specialized purely in carrying out a single well-defined step handed to it by a planner.	Runs the specific "provision the storage account" step and reports back
18. Human-in-the-loop Agent	Pauses at defined checkpoints for human approval before continuing or acting.	Drafts a wire-transfer request but requires human sign-off before execution

INTERVIEW Q&A

Q. What's the difference between a Router Agent and a Supervisor Agent? They sound similar.

A. A router makes a one-time classification decision and hands off completely; it never sees the result. A supervisor stays involved: it delegates, receives results back, and decides the next delegation, potentially to multiple agents in sequence or parallel. Router is stateless per request; supervisor owns the whole trajectory.

Q. Why separate Planner and Executor into different agents instead of one agent doing both?

A. Different failure tolerances and different prompt shapes. A planner benefits from a broader, more exploratory prompt; an executor benefits from a narrow, tightly-scoped one focused on getting one action right. Splitting them also makes it possible to swap a cheaper model into the executor role while keeping a stronger model on planning.

Tip: In a system-design interview, naming Critic and Human-in-the-loop agents unprompted for anything touching money, health, or legal outcomes is a strong signal — it shows you default to safety controls rather than needing to be asked.

SCENARIO PREP

Scenario: Asked why you'd add a Critic agent to a system that 'already works.'

Approach: Frame it as independence rather than redundancy. A Critic catches what the generating agent is structurally biased to miss, and that matters most precisely when things already seem to work.

16 Agentic Design Patterns (1–8)

PATTERN	EXECUTION FLOW	WHEN TO USE / WHEN NOT
1. ReAct	Thought → Action → Observation, repeated per step	Use: general-purpose tool use. Not: latency-critical paths — every step is a full LLM round trip.
2. Plan-and-Execute	Full plan generated once → steps executed → replan only on failure	Use: predictable multi-step tasks. Not: highly exploratory tasks where step 1's result should reshape step 3.
3. Reflection	Draft → self-critique → revise → return	Use: quality-sensitive single outputs. Not: simple factual lookups — doubles the cost for no benefit.
4. Critic	Generator produces output → separate critic model/call scores it → accept or send back	Use: when you want an independent, less-biased check. Not: when latency budget can't absorb a second full call.
5. Router	Classify request → forward to matching handler/agent	Use: request types are cleanly separable. Not: when categories overlap heavily — misrouting compounds downstream.
6. Supervisor	Central agent delegates to specialists → collects → synthesizes	Use: genuinely distinct sub-domains. Not: as a default multi-agent starting point — try one agent first.
7. Hierarchical Agents	Supervisor of supervisors — teams of agents, each with their own sub-supervisor	Use: large orgs of agents (10+) needing structured delegation. Not: small systems — adds layers of latency for no gain under ~5 agents.
8. Sequential Agents	Agent A's output is Agent B's input, in a fixed order	Use: pipeline-shaped work (extract → validate → summarize). Not: when steps could run independently — wastes parallel time.

SEQUENTIAL VS. PARALLEL, VISUALLY



Interview tip: When asked "how would you speed this agent up," check whether any of its sequential steps are actually independent — converting sequential to parallel is often a bigger latency win than swapping models.

SCENARIO PREP

Scenario: Asked to speed up a slow sequential agent, live.

Approach: Check for independent steps before touching the model at all. Parallelizing is usually the bigger, cheaper win compared with swapping in a faster model.

16 Agentic Design Patterns (9–16)

PATTERN	EXECUTION FLOW	WHEN TO USE / WHEN NOT
9. Parallel Agents	Independent agents run concurrently → results merged	Use: sub-tasks with no dependency (check 3 data sources at once). Not: when order affects correctness.
10. Map-Reduce Agents	Same agent applied to N chunks in parallel (map) → results combined by a reducer step	Use: summarizing/analyzing large document sets. Not: small inputs — the reduce overhead isn't worth it.
11. Human-in-the-loop	Agent pauses at a checkpoint, waits for approval/edit, then resumes	Use: irreversible or high-risk actions. Not: high-volume, low-risk steps — kills throughput.
12. Generator-Critic	One agent generates, a second (often different model) critiques against explicit criteria, loop until it passes	Use: content quality gates. Not: open-ended creative work where "correctness" isn't well defined.
13. Evaluator-Optimizer	Like Generator-Critic but the evaluator returns a score, and the loop continues until the score crosses a threshold or a step cap	Use: tasks with a measurable quality signal (code passing tests, SQL returning expected shape). Not: subjective quality with no scorable signal.
14. Agent-as-a-Tool	A whole agent is exposed to another agent as a callable tool, hiding its internal loop	Use: composing specialist agents into a larger system without the caller knowing internals. Not: when the caller genuinely needs visibility into intermediate steps.
15. Tool-as-an-Agent	A traditionally static tool (e.g. a SQL query builder) is wrapped with its own small reasoning loop to self-correct malformed queries	Use: tools with a non-trivial failure/retry surface. Not: simple deterministic tools — adds pointless latency.
16. Event-driven Agents	Agent is triggered by an event (webhook, queue message, file upload) rather than a synchronous user request	Use: background processing (new transaction → fraud-check agent fires). Not: interactive, latency-sensitive user-facing flows.

SUPERVISOR PATTERN — MINIMAL SHAPE

```
def supervisor(goal, agents: dict) -> str:
    plan = llm.decide_delegation(goal, list(agents.keys()))
    results = {}
    for step in plan:                                # sequential; swap to asyncio.gather for parallel
        results[step.agent] = agents[step.agent].run(step.sub_goal)
    return llm.synthesize(goal, results)
```

Interview tip: Generator-Critic and Evaluator-Optimizer get confused constantly. The tell: Critic gives qualitative feedback ("this misses X"), Optimizer gives a quantitative score the loop can check against a threshold programmatically.

SCENARIO PREP

Scenario: Asked the difference between two patterns you always mix up under pressure.

Approach: For Generator-Critic vs. Evaluator-Optimizer, lead with the tell: qualitative feedback vs. a numeric score against a threshold.

Agent & Orchestration Frameworks

Every framework below solves the same core problem — running the agent loop — with a different opinion on how much structure to impose.

LIBRARY	WHAT IT DOES FOR AN AGENTIC SYSTEM	WHERE IT FITS / TRADE-OFF
LangChain	Standardized interfaces for models, prompts, tools, and retrievers; the most common entry point for tool-calling agents.	Fastest way to wire an agent together. Trade-off: its abstractions can hide what's actually happening in the prompt, which hurts debugging if you don't peek under the hood.
LangGraph	Graph-based orchestration with explicit state, cycles, and checkpoints, built by the LangChain team for stateful multi-step agents.	The right layer once an agent needs loops, retries, or human-in-the-loop pauses that plain LangChain chains handle awkwardly.
LlamaIndex	Indexing and retrieval-first framework; strong at connecting many data sources into a queryable index.	Best when the project is retrieval-heavy first, agent-behavior second — the reverse emphasis from LangGraph.
CrewAI	Role-based multi-agent orchestration — define agents by role, goal, and backstory, then hand them a crew of tasks.	Quick to prototype multi-agent demos. Trade-off: less low-level control than hand-rolling a LangGraph supervisor.
AutoGen / AG2	Conversation-driven multi-agent framework where agents "talk" to each other in a shared message thread.	Good fit for research/exploratory multi-agent setups; needs more guardrails before production use than a supervisor pattern.
Semantic Kernel	Microsoft's orchestration SDK with strong C#/.NET support alongside Python; plugin-based tool model.	Natural choice in Azure/.NET-heavy enterprise shops; less community momentum in pure-Python shops than LangChain.

How I actually choose: LangGraph for anything with real state and cycles (most production agents), LlamaIndex when the hard problem is data ingestion and indexing rather than orchestration, CrewAI or AutoGen for fast multi-agent prototypes I don't plan to ship as-is.

INTERVIEW Q&A

Q. Why would you choose LangGraph over plain LangChain for a production agent?

A. Plain LangChain's agent executor is a black-box loop, hard to pause, inspect mid-run, or resume after a crash. LangGraph makes the state explicit and gives you checkpointing, so a step failure doesn't mean starting the whole task over, and a human-approval pause is a first-class node instead of a workaround.

SCENARIO PREP

Scenario: Asked 'why not just use one framework for everything?'

Approach: Answer with the actual trade-off you'd weigh, LangGraph for state and cycles, LlamaIndex for retrieval-heavy work, rather than naming a personal favorite. That's the senior signal.

Models, Retrieval, Data & Observability

MODEL PROVIDER SDKS

PROVIDER	AGENTIC-RELEVANT STRENGTH
OpenAI SDK	Mature structured-output and function-calling support; broad tool ecosystem
Anthropic SDK	Strong long-context reasoning and tool-use reliability for multi-step plans
Google Gemini SDK	Native multimodal tool calls; tight Vertex AI integration
AWS Bedrock	Multi-model access behind one API, inside your AWS network boundary
Azure OpenAI	Enterprise auth/compliance wrapper around OpenAI models for regulated industries

DATA & API LAYER

LIBRARY	AGENTIC ROLE
Pandas / NumPy	Inside tool functions for data-analysis agents
SQLAlchemy	Safe, parameterized DB access for a SQL-agent tool
FastAPI	Exposes the agent as an HTTP service with async support
httpx / requests	Calling external tool APIs from within an agent's tool functions
Pydantic	Validates tool arguments and structured LLM output before execution

VECTOR DATABASES (RAG LAYER)

STORE	WHERE IT'S STRONG
FAISS	In-memory, no server — fast local prototyping
Chroma	Lightweight, easy local + small-team deployments
Qdrant	Strong filtering + hybrid search at scale
Pinecone	Fully managed, minimal ops overhead
Weaviate	Built-in hybrid search and modular embedding backends
Milvus	High-throughput, large-scale self-hosted deployments

OBSERVABILITY & EVALUATION

TOOL	AGENTIC ROLE
LangSmith	Trace every LLM call, tool call, and token in a LangChain/LangGraph agent
Langfuse	Open-source tracing + evaluation, framework-agnostic
OpenTelemetry	Standard tracing spec — ties agent traces into existing infra observability
MLflow	Experiment tracking for prompt/model variants and eval runs

Interview tip: Don't just name libraries — state the trade-off you'd weigh. "Pinecone over FAISS because I don't want to own vector-index ops" is a stronger answer than reciting the feature list.

SCENARIO PREP

Scenario: Asked to justify a vector DB choice for a hypothetical system.

Approach: Anchor the answer to operational burden, not features. 'Pinecone, because I don't want to run index infrastructure' beats reciting a feature table.

LangChain for Agentic AI — Building Up

Five progressive examples. Each one adds exactly one new capability so you can see precisely where the "agentic" behavior enters.

EXAMPLE 1 — SIMPLE LLM CALL (NOT YET AN AGENT)

```
# pip install langchain langchain-anthropic
from langchain_anthropic import ChatAnthropic

llm = ChatAnthropic(model="claude-sonnet-4-6") # reads ANTHROPIC_API_KEY from env
response = llm.invoke("Summarize the risks of a 90% loan-to-value mortgage.")
print(response.content)
```

No loop, no tool, no decision — a single generation. This is the Tier 1 baseline everything else builds on.

EXAMPLE 2 — LLM + ONE TOOL (MANUAL WIRING)

```
from langchain_core.tools import tool

@tool
def get_exchange_rate(base: str, quote: str) -> float:
    "Look up a live currency exchange rate."
    # real implementation would call an FX API
    return 1.09 # placeholder

llm_with_tools = llm.bind_tools([get_exchange_rate])
msg = llm_with_tools.invoke("What's USD to EUR right now?")
print(msg.tool_calls) # the model requests the call; we execute it ourselves below
```

The model now *decides* whether a tool call is needed and generates structured arguments — but we're still manually executing the call and feeding the result back. That manual wiring is what Example 3 automates.

Line-by-line: `@tool` turns a plain function into something the model can call by inspecting its docstring and type hints as the schema — get the docstring wrong and the model will misuse the tool, since that's the only spec it sees.

SCENARIO PREP

Scenario: Asked to explain your own code sample from memory, live.

Approach: Narrate what changes at each step: plain call, then bound tools, then an automatic loop. That progression is the real answer to 'how does tool calling work.'

LangChain — Tool-Calling Agent & RAG

EXAMPLE 3 — TOOL-CALLING AGENT (THE LOOP IS AUTOMATIC NOW)

```
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a banking operations assistant. Use tools when you need live data."),
    ("human", "{input}"),
    ("placeholder", "{agent_scratchpad}"), # where the loop's reasoning trace lives
])
agent = create_tool_calling_agent(llm, [get_exchange_rate], prompt)
executor = AgentExecutor(agent=agent, tools=[get_exchange_rate], max_iterations=6)

result = executor.invoke({"input": "Convert 500 USD to EUR."})
print(result["output"])
```

What changed: AgentExecutor now owns the loop — call model, check for a tool call, execute it, feed the result back, repeat until the model returns a final answer or max_iterations is hit. This is the actual "agent" boundary.

EXAMPLE 4 — AGENT + RAG (RETRIEVER AS A TOOL)

```
from langchain_community.vectorstores import Chroma
from langchain_core.tools import create_retriever_tool

vectorstore = Chroma(persist_directory="./policy_db", embedding_function=embeddings)
retriever_tool = create_retriever_tool(
    vectorstore.as_retriever(search_kwargs={"k": 4}),
    name="search_lending_policy",
    description="Search internal lending policy documents for compliance rules.",
)
agent = create_tool_calling_agent(llm, [get_exchange_rate, retriever_tool], prompt)
```

Retrieval is now just another tool the agent chooses to call — not a fixed pre-step. That's the difference between "RAG app" and "agent with RAG."

INTERVIEW Q&A

Q. What does max_iterations actually protect against?

A. Infinite or excessively long tool-calling loops when the model can't converge on a final answer. Without it, a confused agent can burn tokens and time indefinitely. In production I pair it with a timeout and log a specific "max iterations hit" event for monitoring, not just a silent cutoff.

SCENARIO PREP

Scenario: Asked what happens if max_iterations is hit mid-task.

Approach: Don't just say 'it stops.' Describe the production handling: log a distinct event, return partial progress, alert someone. It should never fail silently.

Why LangGraph Exists

A plain LangChain AgentExecutor is a hidden while-loop — you can't easily pause it, resume it after a crash, or let a human approve one step in the middle. LangGraph makes the loop a graph you define explicitly, with state that persists between steps.

CONCEPT	WHAT IT IS
Graph	The overall structure of nodes and edges representing the agent's possible execution paths.
State	A typed object (often a TypedDict) passed between nodes, carrying messages, tool results, and any custom fields.
Nodes	Functions that read state and return updates — "call the LLM," "run this tool," "check policy."
Edges	Fixed transitions from one node to the next.
Conditional edges	A function inspects the current state and picks which node runs next — this is where branching/looping logic lives.
Checkpoints	Saved snapshots of state after each node, enabling resume-from-failure instead of restart-from-scratch.
Persistence	Checkpoints written to a durable store (Postgres, SQLite, Redis) so state survives process restarts.
Interrupts	A deliberate pause point in the graph — the mechanism behind human-in-the-loop approval.
Cycles	Edges that loop back to an earlier node — how "re-plan and try again" is expressed structurally.

LANGCHAIN AGENT EXECUTOR VS. LANGGRAPH

AGENTEXECUTOR Fast to set up. Loop and state are implicit. No native pause/resume. Fine for simple, short-lived tool-calling agents.	LANGGRAPH More setup, explicit state and transitions. Native checkpointing, interrupts, and cycles. What you reach for once an agent needs to survive a restart or pause for approval.
--	--

Interview tip: If asked "when would you NOT use LangGraph," the honest answer is: a single-shot tool call with no need to resume or branch — the explicit state machine is overhead you don't need yet.

INTERVIEW Q&A

- Q. What actually gets persisted in a checkpoint — the whole conversation, or something smaller?**

A. The full state object at that point in the graph: messages, tool results, and any custom fields you've defined, not just the chat text. That's what makes an exact resume possible instead of a rough replay.
- Q. Are conditional edges evaluated by the LLM or by your own code?**

A. Your own code. A conditional edge is a plain Python function that inspects state and returns a string key. The LLM influences what's *in* state (e.g., whether it requested a tool), but the routing decision itself is deterministic code, which is exactly why it's debuggable.

SCENARIO PREP

Scenario: Asked when you would NOT reach for LangGraph.

Approach: Say it plainly: a single-shot tool call with no resume/branch need doesn't justify the extra state-machine setup. Knowing the boundary beats defaulting to the newer tool.

LangGraph — Project: Agent + Tool + Checkpoint

```
# pip install langgraph langchain-anthropic
from typing import TypedDict, Annotated
from langgraph.graph import StateGraph, END
from langgraph.graph.message import add_messages
from langgraph.checkpoint.memory import MemorySaver

class AgentState(TypedDict):
    messages: Annotated[list, add_messages]

def call_model(state: AgentState):
    response = llm_with_tools.invoke(state["messages"])
    return {"messages": [response]}

def should_continue(state: AgentState) -> str:
    last = state["messages"][-1]
    return "tools" if getattr(last, "tool_calls", None) else END

builder = StateGraph(AgentState)
builder.add_node("agent", call_model)
builder.add_node("tools", tool_node) # prebuilt ToolNode([get_exchange_rate])
builder.set_entry_point("agent")
builder.add_conditional_edges("agent", should_continue, {"tools": "tools", END: END})
builder.add_edge("tools", "agent") # the cycle: tool result goes back to the model

graph = builder.compile(checkpointer=MemorySaver(), interrupt_before=["tools"])
config = {"configurable": {"thread_id": "case-4471"}}
graph.invoke({"messages": [("user", "Convert 500 USD to EUR")]}, config)
```

Line-by-line: the conditional edge checks whether the model's last message requested a tool; if so it routes to the `tools` node, otherwise it ends. `interrupt_before=["tools"]` pauses execution right before any tool runs — a human reviewing `graph.get_state(config)` can approve or edit the pending call before it executes. `thread_id` is what makes the checkpoint resumable later.

SUPERVISOR + MULTIPLE AGENTS, IN GRAPH FORM

```
builder = StateGraph(SupervisorState)
builder.add_node("supervisor", supervisor_node) # decides which specialist runs next
builder.add_node("fraud_agent", fraud_node)
builder.add_node("policy_agent", policy_node)
builder.add_conditional_edges("supervisor", route_fn,
    {"fraud": "fraud_agent", "policy": "policy_agent", "done": END})
builder.add_edge("fraud_agent", "supervisor") # specialists report back, supervisor decides next
builder.add_edge("policy_agent", "supervisor")
```

Interview tip: Point out that specialists always report back *to the supervisor*, never directly to each other — that single rule is what prevents the deadlock and duplicate-work failure modes covered on page 31.

SCENARIO PREP

Scenario: Asked to add a human-approval step to a graph you didn't design.

Approach: Name the mechanism specifically: `interrupt_before` on the risky node, not human-in-the-loop in the abstract. Specifics read as real experience.

Agent Memory

SHORT-TERM

The current run's message trace and tool results. Lives in the LangGraph state object; gone when the run ends unless checkpointed.

LONG-TERM

Persists across sessions — user preferences, prior decisions. Backed by a database or vector store keyed by user/thread ID.

CONVERSATION MEMORY

Raw or summarized chat history. Summarized once the trace grows past a token budget, to avoid context bloat.

EPISODIC MEMORY

Specific past events ("last time, the retry on this tool failed twice") the agent can recall to avoid repeating mistakes.

SEMANTIC MEMORY

General facts and preferences distilled from many episodes ("this user always wants amounts in EUR").

WORKING MEMORY

The immediate scratchpad within a single reasoning step — intermediate values the model is actively using right now.

STORAGE CHOICE	FITS WHICH MEMORY TYPE
Key-value store (Redis)	Fast short-term state, session-scoped conversation memory
Relational DB (Postgres)	LangGraph checkpoints, structured long-term facts per user
Vector DB	Semantic memory retrieved by similarity ("what do we know that's relevant to this?")
Summarization via LLM call	Compressing long conversation history into a durable summary

MEMORY RISKS

MEMORY POISONING

An attacker or a bad tool result writes false "facts" into long-term memory that later bias unrelated answers.

PRIVACY & DATA LEAKAGE

Memory scoped to the wrong key can leak one user's data into another user's session.

CONTEXT EXPLOSION

Unsummarized history grows until it crowds out the actual task in the context window.

STALE / INCORRECT MEMORY

A fact that was true when stored (an old account balance) is trusted long after it's changed.

INTERVIEW Q&A

Q. How would you prevent memory poisoning in a production agent?

A. Never write raw tool output directly to long-term memory unvalidated. Pass it through a filter step that checks it against expected schema and plausibility, and version memory writes so a bad entry can be rolled back once detected. I also scope what's allowed into long-term memory at all; most tool results should stay short-term only.

Q. When would you deliberately choose NOT to give an agent long-term memory?

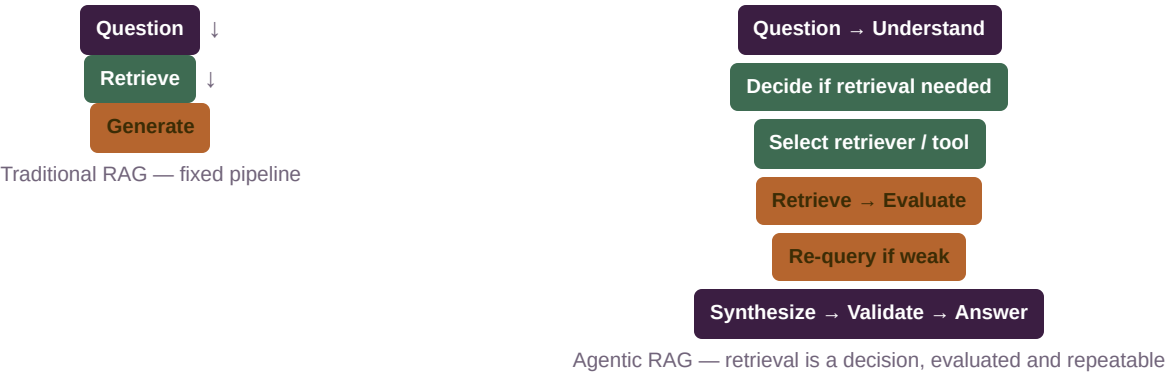
A. Anything handling sensitive one-off transactions where remembering creates more privacy risk than personalization benefit. A public-facing FAQ bot doesn't need to remember individual users across sessions, and adding that memory just adds a data-retention liability.

SCENARIO PREP

Scenario: Asked how you'd fix an agent 'forgetting' something a user told it earlier.

Approach: Diagnose before fixing. Is short-term state failing to pass forward, or is this a long-term memory gap? The fix looks completely different depending on which one it is.

Traditional RAG vs. Agentic RAG



TECHNIQUE	WHAT IT SOLVES
Query planning	Splits a compound question into retrievable sub-questions before searching.
Query rewriting	Reformulates a vague user question into a retrieval-friendly query.
Multi-query retrieval	Generates several phrasings of the same question to widen recall.
Hybrid search	Combines keyword (BM25) and vector similarity — catches exact terms embeddings miss.
Metadata filtering	Restricts retrieval by date, department, or access level before similarity search runs.
Reranking	A second, more expensive model re-scores the top-k results for relevance before they reach the generator.
Retrieval evaluation	Checks retrieved chunks actually support the question before generating — if not, triggers re-query.
Self-correction	The agent notices its own answer isn't grounded in the retrieved context and retries with a different query.

Interview framing: "Traditional RAG always retrieves once and generates. Agentic RAG treats retrieval as fallible — it checks whether what it found actually answers the question before committing to an answer."

INTERVIEW Q&A

- Q. Why does query rewriting matter if the vector search already handles semantic similarity?**
A. Embeddings capture meaning, not phrasing style, so a vague or jargon-mismatched query can still miss relevant chunks that use different wording. Rewriting closes that gap before the retriever even runs.
- Q. When would hybrid search outperform pure vector search?**
A. Whenever exact terms matter: account numbers, error codes, regulation names. Embeddings optimize for semantic similarity, not lexical match; BM25 alongside vector search catches what pure similarity search drops.

SCENARIO PREP
Scenario: Asked why your RAG system sometimes 'gives up' and asks the user to rephrase.
Approach: That's the retry cap doing its job. Explain the groundedness-check-then-retry loop instead of apologizing for it like a bug.

Agentic RAG — Multi-Source with Self-Correction

```
def agentic_rag(question: str, vector_retriever, sql_tool, llm, max_retries=2) -> str:
    source = llm.decide_source(question)          # "vector", "sql", or "both"
    attempt, context = 0, ""
    while attempt <= max_retries:
        if source in ("vector", "both"):
            context += vector_retriever.search(question, k=5)
        if source in ("sql", "both"):
            context += sql_tool.query(llm.to_sql(question))
        grounded = llm.check_groundedness(question, context) # retrieval evaluation
        if grounded.score >= 0.7:
            break
        question = llm.rewrite_query(question, why_failed=grounded.reason) # self-correction
        attempt += 1
    answer = llm.generate(question, context)
    if not llm.check_groundedness(question, answer).passed:
        return "I don't have enough verified information to answer confidently."
    return answer
```

What's agentic here: the source decision, the groundedness check, and the rewrite-and-retry loop. A plain RAG function would stop after the first retrieve-then-generate pass regardless of quality.

SQL + VECTOR, COMBINED

A common banking pattern: "What's our exposure to clients in default in the Northeast region?" needs a SQL aggregate (exposure numbers) *and* a vector search (definition of "default" per current policy). Agentic RAG decides both are needed and merges results rather than forcing the question into one retriever.

INTERVIEW Q&A

Q. How do you evaluate "groundedness" programmatically, not just by eyeballing outputs?

A. An LLM-as-judge call that's given the answer and the retrieved context and asked to output a score plus which claims aren't supported. That score then gets logged per request, so retrieval quality is trackable over time instead of checked ad hoc.

Q. What's the failure mode of setting the groundedness threshold too high?

A. The agent gets stuck re-querying on legitimately hard questions where good-enough context simply doesn't exist, burning latency and cost until it hits `max_retries` and returns nothing useful anyway. Threshold and retry cap have to be tuned together against real failure data, not set independently.

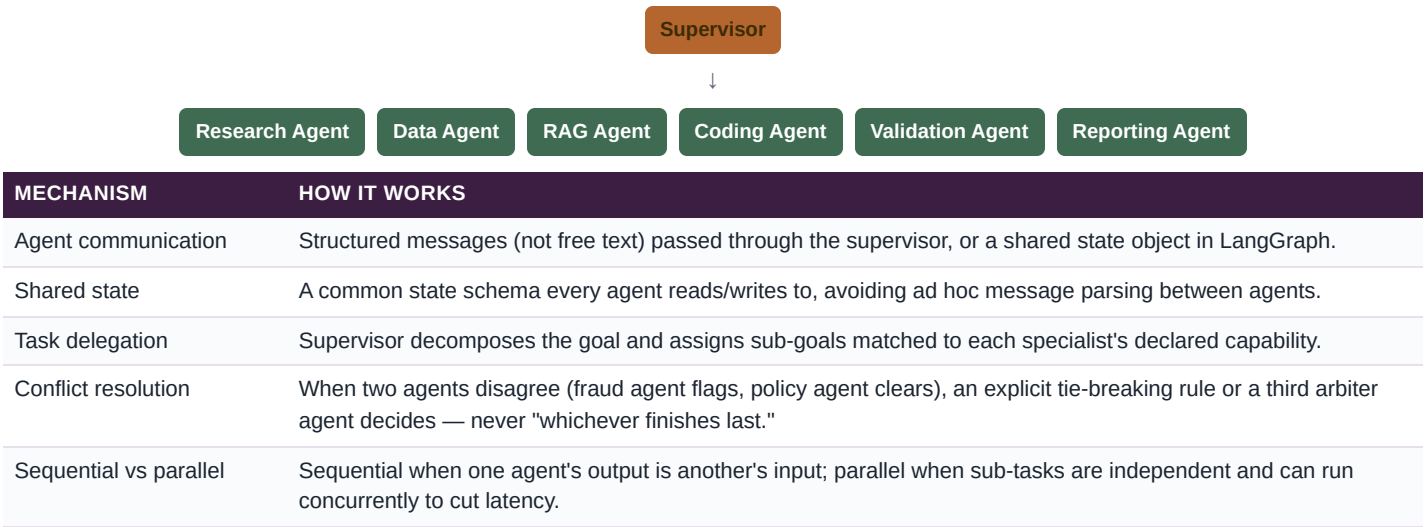
SCENARIO PREP

Scenario: Asked what you'd change if retrieval stays poor on a query type even after retries.

Approach: Move past prompt tuning. Check the chunking strategy first, and whether the source needs metadata filtering or a completely different retriever.

Multi-Agent Systems

Reach for multiple agents when a task has genuinely separable sub-domains that need different tools, different prompts, or different models — not because "multi-agent" sounds more advanced.



COST AND LATENCY REALITY

Each specialist agent is its own set of LLM calls. A 4-agent system answering one user question can easily mean 6–10 LLM calls once you include the supervisor's delegation and synthesis calls — that's a real 4–6x cost multiple over a single well-scoped agent, and it compounds if any specialist itself loops.

When NOT to use multi-agent: the task doesn't decompose into genuinely independent sub-domains, latency budget is tight (each hop adds a round trip), or a single agent with a focused toolset already handles it reliably. Multi-agent is a scaling tool for complexity, not a default architecture.

INTERVIEW Q&A

- Q. How do you prevent two specialist agents from doing the same work twice?**
A. The supervisor tracks which sub-goals have been delegated and to whom in shared state, and specialists check that state before acting rather than blindly executing whatever they're handed. Duplicate work is usually a sign the delegation record isn't being consulted.
- Q. Real-world example from banking: why split fraud detection and policy lookup into separate agents instead of one?**
A. Fraud detection benefits from a model tuned toward sensitivity (flag more, verify later) with tools like transaction-pattern analysis, while policy lookup needs precision (cite the exact clause) with a RAG toolset over compliance documents. Merging them into one prompt makes the model split attention between two very different reasoning styles, and in practice that's where I've seen tool-selection accuracy drop the most.

SCENARIO PREP

Scenario: Asked to estimate cost for a proposed 5-agent design on the spot.

Approach: Ballpark it from the rule of thumb on p.26 (6–10+ LLM calls per request), and say plainly that you'd instrument real numbers before committing to anything more precise.

Tool Calling

A tool is a function; a schema is how the model knows it exists. Function calling is the model's structured request to invoke one; tool calling is the runtime executing it and returning the result as an observation.

1. CALCULATOR

Deterministic math the model shouldn't approximate.

2. WEATHER API

Live external data the model can't know.

3. SQL DATABASE

Structured internal data lookups — always parameterized, never raw string interpolation.

4. SEARCH

Web or internal document search for current information.

5. FILE READER

Reads an uploaded or referenced document into context.

6. PYTHON EXECUTION

Sandboxed code execution for data transforms — never unsandboxed.

```
from pydantic import BaseModel, Field

class TransferArgs(BaseModel):
    account_id: str = Field(description="Source account, format ACCT-XXXXXX")
    amount_usd: float = Field(gt=0, le=5000, description="Amount, capped per policy")

@tool(args_schema=TransferArgs)
def initiate_transfer(account_id: str, amount_usd: float) -> str:
    "Initiate an internal transfer. Requires human approval above $1000."
    if amount_usd > 1000:
        return queue_for_approval(account_id, amount_usd) # never auto-executes above threshold
    return execute_transfer(account_id, amount_usd)
```

Secure by construction: the schema itself caps the amount ($le=5000$), and the function enforces a second, lower threshold for unattended execution — two independent controls, not one.

WHY AN AGENT SHOULD NOT HAVE UNLIMITED TOOLS

TOOL CONFUSION

Past ~15–20 tools, selection accuracy measurably drops — the model starts picking plausible-sounding wrong tools.

SECURITY & COST

Every tool is an attack surface and a latency/cost line item, whether or not it's ever called that session.

HALLUCINATED CALLS

A model can emit a call to a tool that doesn't exist or with args that don't match the schema — always validate before executing.

INCORRECT ACTIONS

A right tool with wrong arguments is often worse than no tool call at all — it looks successful but does the wrong thing.

INTERVIEW Q&A

Q. How do you decide the threshold for requiring human approval on a tool like this transfer example?

A. Match it to actual organizational risk appetite, not an arbitrary round number, and it's often set by referencing existing manual-review thresholds the business already uses elsewhere, so the agent's automation boundary lines up with policy that's already been approved rather than inventing a new one.

SCENARIO PREP

Scenario: Asked what you'd do if a tool call succeeded but with subtly wrong arguments.

Approach: Point out that this is often worse than a failed call. Schema validation won't catch arguments that are semantically wrong but technically valid; that needs a separate check.

Agent Security — Threat Landscape

Agents expand the attack surface of an LLM app because they can *act*, not just generate text. Every threat below assumes the attacker can influence input somewhere in the pipeline.

THREAT	HOW IT HAPPENS
Prompt injection	User input directly instructs the model to ignore its system prompt or safety rules.
Indirect prompt injection	Malicious instructions hidden in a retrieved document, webpage, or email the agent reads — the user never typed them.
Tool injection	A tool's own output contains text crafted to manipulate the next reasoning step.
Data exfiltration	The agent is tricked into including sensitive context in an outbound call (e.g., an email tool) it has legitimate access to.
Excessive agency	The agent has broader tool permissions than the task requires, so a successful injection can do more damage.
Privilege escalation	Chained tool calls let the agent reach data or actions beyond its intended scope.
Credential leakage	API keys or secrets end up in a prompt, a log, or a generated response.
Malicious documents	A poisoned PDF or webpage in the retrieval corpus carries injected instructions.
SSRF	A URL-fetching tool is tricked into calling an internal-only endpoint the attacker couldn't reach directly.
API abuse	An agent with unrestricted tool-call volume is used to hammer a downstream API past its rate limits.

Interview tip: Indirect prompt injection is the one most candidates miss. If your agent's RAG corpus or web-browsing tool can pull in attacker-controlled content, that's an injection vector even if your users are fully trusted.

INTERVIEW Q&A

- Q. How is excessive agency different from a straightforward permissions bug?**

A. A permissions bug is a mistake; excessive agency is a design default. The agent was deliberately given broad tool access "to be safe" or "for flexibility." The fix isn't patching a bug, it's re-scoping what the agent can reach at all, task by task.
- Q. Why is SSRF a distinctly agent-shaped risk, not just a general web security issue?**

A. Because the agent decides which URL to fetch based on its own reasoning, which an attacker can influence indirectly through the content it's processing. A traditional app's URL inputs are usually far more constrained and explicit.

SCENARIO PREP

Scenario: Asked which security risk you think junior candidates most overlook.

Approach: Indirect prompt injection. Most people only defend against what the user types and forget that a retrieved document or tool result can carry instructions too.

Agent Security Architecture

AUTHENTICATION & AUTHORIZATION

Every tool call carries the calling user's identity; the tool itself checks permission, not just the agent's say-so.

RBAC & LEAST PRIVILEGE

Each agent/tool combination gets the minimum scope needed — a support agent's DB credential can read tickets, not write payroll.

TOOL PERMISSIONS

Explicit allow-lists per agent role; no agent gets "all tools" by default.

INPUT / OUTPUT VALIDATION

Schema-validate tool arguments before execution and scan generated output for secrets or policy violations before it leaves the system.

SANDBOXING

Code execution and file access tools run in an isolated environment with no access to production credentials or the host filesystem.

SECRETS MANAGEMENT

Credentials live in a vault (not the prompt, not the codebase); tools fetch them at call time and never pass them through the LLM context.

AUDIT LOGGING

Every tool call, argument, and result is logged with the initiating user and request ID for forensic replay.

HUMAN APPROVAL

Irreversible or high-value actions pause for explicit sign-off — implemented as a LangGraph interrupt in practice.

INTERVIEW Q&A

Q. How would you defend against indirect prompt injection from a RAG corpus you don't fully control?

A. Treat retrieved content as data, never as instructions. Wrap it clearly in the prompt as "reference material" the model should not treat as commands, strip or flag suspicious imperative language during ingestion, and keep the tool-permission surface narrow enough that even a successful injection can't do much damage.

Q. Scenario: an agent's log shows it called a "send_email" tool with the full customer database attached. What do you check first?

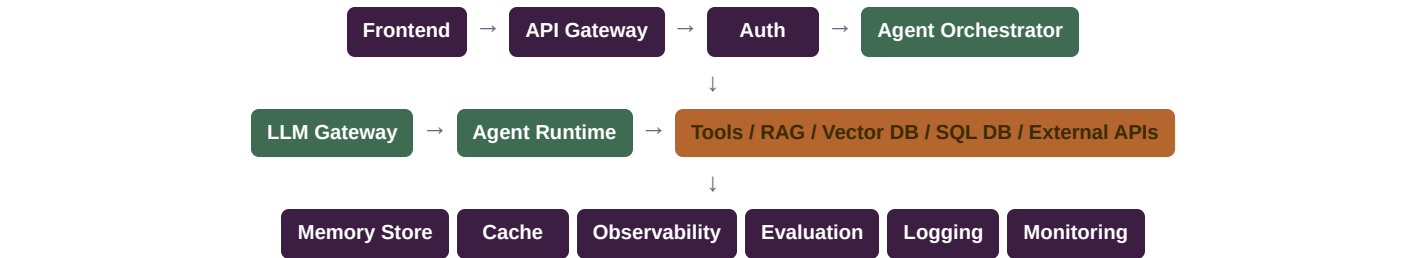
A. Whether that tool call was even a valid path for the task the user requested. If not, this points to excessive agency (the tool shouldn't have been reachable for this task) rather than a one-off model mistake, and I'd tighten the tool's allow-list and add an output-size cap before re-enabling it.

SCENARIO PREP

Scenario: Asked how you'd respond to a live incident where an agent leaked data.

Approach: Contain first (revoke the tool's scope), then investigate the trace for injection vs. excessive agency. Don't jump to a prompt patch before knowing the cause.

Production Agent Architecture



LAYER	PRODUCTION ROLE
API Gateway	Rate limiting, request validation, and the single ingress point — nothing reaches the orchestrator unauthenticated.
LLM Gateway	Abstracts which underlying model serves a request; enables model routing and centralized cost tracking.
Agent Runtime	The compiled LangGraph (or equivalent) executing the loop, with checkpointing for resumability.
Cache	Stores results for repeated/similar queries — often the single biggest cost lever available.
Memory Store	Durable short/long-term state, separate from the transient cache.
Observability / Evaluation / Logging / Monitoring	Four related but distinct concerns covered in full on pages 32–34.

Interview tip: Draw this exact stack when asked to "design a production RAG/agent system" — then narrow to what the specific prompt actually needs. Interviewers want to see you know the full picture before you simplify it.

SCALABILITY & HIGH AVAILABILITY

HORIZONTAL SCALING

Every layer above should be stateless and horizontally scalable except the memory store and vector DB, which scale by sharding/replication instead of just adding instances.

HIGH AVAILABILITY

Multi-region LLM gateway failover, replicated checkpoint store, and graceful degradation — e.g., serve a cached or simplified answer rather than a hard failure when the vector DB is unreachable.

INTERVIEW Q&A

Q. What's the hardest part of this stack to scale, and why?

A. The vector DB under high write volume. Unlike the stateless orchestration layers, it can't just be replicated arbitrarily without index consistency trade-offs. In practice I've planned around read replicas for query load and accepted slightly delayed indexing for new documents rather than trying to make writes fully real-time at scale.

SCENARIO PREP

Scenario: Whiteboarding a production system and running out of time.

Approach: Draw all 8 layers at a high level first, then go deep on one. A complete but shallow diagram beats a detailed one you didn't finish.

Reliability Patterns for Agents

PATTERN	WHY AN AGENT NEEDS IT SPECIFICALLY
Rate limiting	An agent looping on a bad plan can hammer downstream APIs far faster than a human ever would.
Retries with backoff	Transient tool/API failures shouldn't fail the whole task — but retry the tool call, not the entire agent run.
Timeouts	Every tool call and LLM call needs one; an agent waiting forever on a hung tool blocks the whole loop.
Circuit breakers	If a downstream tool is consistently failing, stop calling it for a cooldown period instead of retrying into a known outage.
Caching	Cache both retrieval results and full agent responses for repeated queries — the highest-leverage cost control available.
Async execution	Independent tool calls or parallel specialist agents run concurrently instead of blocking sequentially.
Queue-based architecture	Long-running agent tasks get queued and processed asynchronously, with the user polling or receiving a webhook, rather than holding an open HTTP connection.
Model routing	Cheap/fast model handles simple classification steps; the expensive model is reserved for the reasoning steps that actually need it.

INTERVIEW Q&A

Q. An agent task takes 45 seconds end to end. How do you make that a good user experience?

A. Move it to a queue-based flow: return immediately with a task ID, stream intermediate status ("searching policy documents...", "checking account history...") over a websocket or polling endpoint, and notify on completion. Users tolerate 45 seconds fine if they can see progress; they don't tolerate a frozen spinner.

Q. Where does model routing save the most money in practice?

A. Classification and routing steps, like "which specialist should handle this" or "does this need retrieval," almost never need the largest model. Routing those to a small, fast model while reserving the frontier model for actual synthesis and complex reasoning steps is often a 30–50% cost reduction with no accuracy loss on the parts that matter.

Key point: Reliability engineering for agents is mostly the same discipline as any distributed system — the new part is that failures can also be *reasoning* failures (bad plan, wrong tool), which none of these infrastructure patterns catch. That's what evaluation (next page) is for.

SCENARIO PREP

Scenario: Asked which reliability pattern you'd add first to an agent with zero safeguards.

Approach: Timeouts, every time. They're the cheapest safeguard to add, and they prevent the single most common failure: a hung call blocking the entire loop.

Agent Evaluation

Evaluating only the final answer misses most of where agents actually fail. A wrong answer can come from a fine plan with bad retrieval, or a perfect retrieval with a broken plan — you need visibility into each stage.

OUTCOME METRICS

- Task success rate
- Groundedness / hallucination rate
- Safety violations

PROCESS METRICS

- Tool selection accuracy
- Planning accuracy
- Retrieval precision/recall
- Number of steps to completion

OPERATIONAL METRICS

- Latency (p50/p95)
- Token usage & cost per request
- Failure / timeout rate

EVALUATION METHODS

- Offline: golden dataset, run before every deploy
- Online: live traffic sampling + LLM-as-judge
- Human evaluation for high-stakes samples
- Regression testing against prior versions

A MINIMAL EVALUATION FRAMEWORK

```
def evaluate_run(trace: AgentTrace, golden: GoldenExample) -> EvalResult:
    return EvalResult(
        task_success = trace.final_answer.matches(golden.expected),
        tool_accuracy = sum(t in golden.expected_tools for t in trace.tools_called) /
max(len(trace.tools_called), 1),
        groundedness = llm_judge.score(trace.final_answer, trace.retrieved_context),
        steps = len(trace.actions),
        cost_usd = trace.total_tokens * MODEL_PRICE_PER_TOKEN,
        latency_ms = trace.end_time - trace.start_time,
    )
```

INTERVIEW Q&A

Q. Why is LLM-as-judge controversial, and how do you mitigate its weaknesses?

A. The judge model can share the same blind spots as the model it's evaluating, and it's sensitive to answer length and phrasing rather than pure correctness. I mitigate this by using a different (often stronger) model as judge than the one being evaluated, anchoring the judge prompt with explicit rubric criteria rather than an open "is this good," and validating judge scores against a human-labeled sample periodically.

Q. What's the minimum evaluation you'd insist on before shipping any agent change?

A. A golden dataset regression run, even just 30–50 representative cases, comparing tool accuracy and task success against the previous version. A prompt tweak that looks like an improvement on the cases you tested by hand can silently regress cases you didn't think to check.

SCENARIO PREP

Scenario: Asked how you'd catch a quality regression that shipped without throwing a single error.

Approach: Describe live groundedness sampling. Error-rate monitoring by itself will never catch a silent quality drop.

Troubleshooting Playbook (1–13)

PROBLEM	LIKELY CAUSE	FIX / PREVENTION
1. Stuck in loop	No progress check between iterations; evaluator never satisfied	Hard step cap + require the plan to change between iterations, not just repeat
2. Calls wrong tool	Ambiguous tool descriptions or overlapping tool purposes	Rewrite descriptions to be mutually exclusive; add few-shot examples in the system prompt
3. Never calls a tool	Model is confident (wrongly) it already knows the answer	System prompt instruction to always verify with a tool when live/current data is implied
4. Calls too many tools	Tool list too large or vague stop condition	Trim toolset; add explicit "stop once you have X" instruction
5. Hallucination	No grounding requirement; model fills gaps confidently	Groundedness check before returning; require citations to retrieved sources
6. Wrong RAG result	Poor chunking or embedding mismatch	Re-chunk with overlap; try hybrid search; add reranking
7. Poor retrieval generally	Query doesn't match document phrasing	Query rewriting / multi-query retrieval before searching
8. Context too large	Unsummarized history or over-retrieval	Summarize older turns; cap retrieved chunk count; truncate tool outputs
9. High token usage	Verbose prompts, redundant context on every step	Trim system prompt; cache static context; use structured output to shorten responses
10. High latency	Sequential steps that could run in parallel; oversized model for simple steps	Parallelize independent calls; route simple steps to a smaller model
11. High cost	No caching; large model used for every step	Cache repeated queries; model routing; cap max steps
12. Memory corruption	Concurrent writes to shared state without locking	Use a single writer per state key or an atomic checkpoint mechanism
13. Incorrect state	State schema mismatch between graph versions	Version state schemas; migrate or reset checkpoints on breaking changes

Debugging approach that applies to all 25: reproduce with the full trace (page 33–34 assumes you have one — if you don't, that's issue zero). Isolate which layer failed — planning, tool call, retrieval, or synthesis — before changing anything.

SCENARIO PREP

Scenario: Given a bug report with no trace attached.

Approach: Ask for the trace before proposing anything. Diagnosing from a text description alone risks fixing the wrong layer entirely.

Troubleshooting Playbook (14–25)

PROBLEM	LIKELY CAUSE	FIX / PREVENTION
14. Graph doesn't terminate	Conditional edge missing an END branch, or cycle with no exit condition	Audit every conditional edge for a guaranteed terminating path; add a hard iteration cap regardless
15. Conditional edge doesn't work	Routing function returns a key not present in the edge mapping	Assert the routing function's return values match the mapping's keys at build time
16. Tool schema mismatch	Model output doesn't conform to the Pydantic schema	Validate and return a structured error back to the model so it can retry with corrected arguments
17. API timeout	Downstream service slow or overloaded	Set explicit per-call timeouts; retry with backoff; circuit-break on repeated failure
18. Rate limit hit	Too many concurrent calls to a provider or tool API	Request queuing, exponential backoff, and provider-side quota monitoring
19. JSON parsing failure	Model output isn't valid JSON despite instructions	Use native structured-output / tool-calling modes instead of asking for JSON in free text
20. Structured output failure	Schema too complex or ambiguous for the model to fill reliably	Simplify the schema; add field-level descriptions; validate and re-prompt on failure
21. Prompt injection observed	Untrusted content treated as instructions	Isolate retrieved/tool content as clearly-labeled data; sanitize before it reaches the prompt
22. Inconsistent answers	High temperature, or retrieval returning different chunks each run	Lower temperature for factual tasks; cache retrieval for identical queries
23. Multi-agent deadlock	Two agents each waiting on the other's output	Supervisor pattern with a single decision point avoids peer-to-peer waiting entirely
24. Duplicate tool execution	Retry logic re-runs a tool call that already partially succeeded	Idempotency keys on tool calls; check state before re-executing
25. Production failure (general)	Usually a combination: an edge case not in the golden dataset plus missing monitoring	Add the case to the golden set, add an alert for that failure signature, and ship a targeted fix — not a broad prompt rewrite

INTERVIEW Q&A

Q. Walk through how you'd debug "the agent keeps calling the same tool repeatedly."

A. Pull the trace and check whether the tool's result is actually changing the state the model sees. Often the result comes back fine but never gets merged into context correctly, so from the model's perspective it never got an answer and reasonably tries again. That's a state-wiring bug, not a prompting problem, and no amount of prompt tuning fixes it.

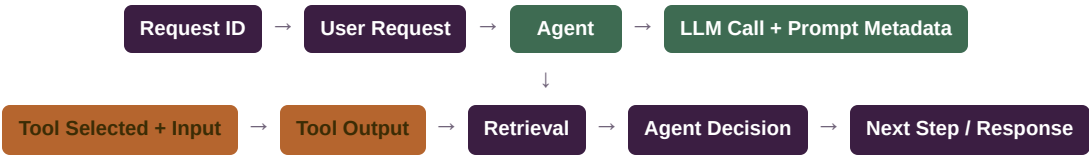
SCENARIO PREP

Scenario: Asked to fix a bug fast without over-engineering the solution.

Approach: Match the fix to the specific failure mode instead of reaching for a broad rewrite. A targeted fix beats 'redesign the prompt' almost every time.

Observability

If you can't reconstruct exactly why an agent produced a specific answer, you can't fix it — only guess. The trace below is the minimum I'd want logged per request.



CATEGORY	WHAT TO CAPTURE
Tracing	Full span tree per request — every LLM call, tool call, and retrieval nested under the parent request span.
Metrics	Aggregate counters: success rate, average steps, tool usage distribution, error rate by type.
Logs	Structured (not free-text) logs keyed by request ID for every decision point, so they're queryable, not just readable.
Token monitoring	Input/output tokens per call, aggregated per request and per user, to catch runaway context growth early.
Cost monitoring	Dollar cost per request, broken down by model and by layer (planning vs. tool calls vs. synthesis).
Latency monitoring	Per-span latency, so you can tell whether a slow request was one slow tool call or many small ones.
Agent trajectory monitoring	The full sequence of decisions for a request, replayable — this is what LangSmith/Langfuse traces are built around.

Interview tip: When asked "how would you debug a bad agent response in production," the strong answer starts with "pull the trace," not "look at the prompt." Prompts are one input; the trace shows what actually happened.

INTERVIEW Q&A

Q. What's the difference between logging and tracing for an agent, and why do you need both?

A. Logs are discrete events ("tool X called with args Y"); a trace connects all the events for one request into a single navigable tree with timing. Logs alone tell you something happened; a trace tells you the order, the timing, and the causal chain. You need the trace to actually debug a multi-step agent, not just confirm that an event fired.

Q. How do you monitor for "silent" quality regressions that don't throw errors?

A. Sample a percentage of live traffic through the same LLM-as-judge groundedness check used offline, and alert when the rolling average score drifts below a baseline. A model or prompt change can degrade answer quality without a single exception being thrown, so error-rate monitoring alone won't catch it.

SCENARIO PREP

Scenario: Asked what you'd instrument first on a brand-new agent with zero observability.

Approach: Request tracing with token and cost per span, before dashboards or alerts. You can't build good alerts on data you aren't capturing yet.

Agentic AI Cost Optimization

LEVER	HOW IT REDUCES COST
Model selection / routing	Route classification and simple steps to a small, cheap model; reserve the frontier model for synthesis and complex reasoning.
Prompt optimization	Trim boilerplate system prompts and few-shot examples that don't change model behavior measurably.
Context reduction	Summarize history, cap retrieved chunk count, truncate tool outputs to what's actually needed.
Caching	Cache identical or near-identical queries and their full responses — usually the single biggest lever.
Retrieval optimization	Retrieve fewer, higher-quality chunks via reranking instead of over-fetching and letting the model sort it out.
Tool optimization	Batch tool calls where possible instead of one call per item.
Limiting agent loops	A tight, well-tuned step cap prevents runaway iterations from ever reaching production cost dashboards.
Parallel execution	Doesn't cut cost directly, but cuts latency, which often lets you use a cheaper, less time-pressured model.
Batch processing	Non-interactive workloads (nightly summarization, bulk classification) run via batch APIs at a fraction of real-time pricing.

Worked example: A transaction-monitoring agent was routing every request — including simple "is this transaction over \$10k" checks — through a large reasoning model with a 6-tool toolkit and no caching. Moving that classification to a small model, capping retrieval to top-3 reranked chunks, and caching repeated account lookups cut cost per request by roughly 60% with no measurable drop in fraud-catch rate, because the expensive model was never actually needed for the classification step.

INTERVIEW Q&A

Q. Your director asks you to cut agent costs 40% without hurting quality. What's your first move?

A. Instrument cost per request broken down by layer before changing anything. You can't optimize what you haven't measured. In most systems I've seen, 60-80% of the wasted spend is concentrated in one or two steps (usually an oversized model on a simple step, or missing caching), not spread evenly, so the fix is targeted, not a blanket "use a smaller model everywhere."

Key point: Cost and latency optimizations are usually the same optimizations — caching, routing, and reducing unnecessary steps improve both simultaneously.

SCENARIO PREP

Scenario: Asked to cut costs with a hard constraint that quality cannot drop.

Approach: Start with caching and model routing rather than model downgrades. Those levers cut cost without touching the reasoning quality that actually matters.

Project A — Basic Tool-Calling Agent

OBJECTIVE

A banking-ops agent that answers balance and transaction questions using two tools, with a visible reasoning trace.

FOLDER STRUCTURE

project_a/agent.py, project_a/tools.py,
project_a/.env, project_a/requirements.txt

```
# requirements.txt
langchain==0.3.*
langchain-anthropic==0.3.*
python-dotenv==1.0.*

# .env (never hard-code the key in source)
ANTHROPIC_API_KEY=sk-ant-...

# tools.py
from langchain_core.tools import tool

@tool
def get_balance(account_id: str) -> float:
    "Return the current balance for an account ID."
    return MOCK_DB.get(account_id, {}).get("balance", 0.0)

@tool
def get_recent_transactions(account_id: str, limit: int = 5) -> list[dict]:
    "Return the most recent transactions for an account."
    return MOCK_DB.get(account_id, {}).get("transactions", [])[:limit]

# agent.py
import os
from dotenv import load_dotenv
from langchain_anthropic import ChatAnthropic
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain_core.prompts import ChatPromptTemplate
from tools import get_balance, get_recent_transactions

load_dotenv()
llm = ChatAnthropic(model="claude-sonnet-4-6", api_key=os.environ["ANTHROPIC_API_KEY"])
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a banking assistant. Always verify with a tool before stating a number."),
    ("human", "{input}"), ("placeholder", "{agent_scratchpad}"),
])
tools = [get_balance, get_recent_transactions]
executor = AgentExecutor(agent=create_tool_calling_agent(llm, tools, prompt), tools=tools, max_iterations=6,
verbose=True)

if __name__ == "__main__":
    print(executor.invoke({"input": "What's the balance on ACCT-10293?"})["output"])
```

Expected output: a verbose trace showing the tool call to `get_balance`, then a final answer citing the actual returned number. **Troubleshooting:** if the agent answers without calling a tool, tighten the system prompt's verification instruction; if it errors on missing key, confirm `.env` is loaded before the client is constructed.

SCENARIO PREP

Scenario: Asked to extend Project A live with one more tool.

Approach: Write the tool's docstring as carefully as the function itself. It's the only spec the model ever sees, and a vague one is the most common cause of tool misuse.

Project B — Agentic RAG over Policy Docs

OBJECTIVE

Answer lending-policy questions from a local document set, retrying with a rewritten query when retrieval is weak.

FOLDER STRUCTURE

project_b/ingest.py, project_b/rag_agent.py,
project_b/docs/, project_b/.env

```
# requirements.txt
langchain==0.3.* langchain-anthropic==0.3.* langchain-chroma==0.2.* chromadb==0.5.*

# ingest.py — run once to build the local vector store
from langchain_community.document_loaders import DirectoryLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_chroma import Chroma
from langchain_anthropic import ChatAnthropic # swap for a real embedding model in prod

docs = DirectoryLoader("./docs", glob="**/*.txt").load()
chunks = RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=120).split_documents(docs)
Chroma.from_documents(chunks, embedding=embeddings, persist_directory="./policy_db")

# rag_agent.py
def answer(question: str, max_retries: int = 2) -> str:
    store = Chroma(persist_directory="./policy_db", embedding_function=embeddings)
    for attempt in range(max_retries + 1):
        docs = store.similarity_search(question, k=4)
        context = "\n\n".join(d.page_content for d in docs)
        grounded = llm.invoke(f"Does this context answer '{question}'? Reply YES or NO.\n\n{context}")
        if "YES" in grounded.content.upper():
            break
        question = llm.invoke(f"Rewrite this query to search better: {question}").content
    return llm.invoke(f"Answer using only this context:\n\n{context}\n\nQuestion: {question}").content

if __name__ == "__main__":
    print(answer("What LTV ratio requires additional collateral review?"))
```

Expected output: an answer citing the specific policy threshold, grounded in retrieved text. **Troubleshooting:** if it always retries to the cap, check chunk size — 800 characters is often too large for dense policy tables; try 400 with more overlap.

INTERVIEW Q&A

Q. Why is the groundedness check here a second LLM call instead of a simple string match?

A. Because "does this context answer the question" is a semantic judgment, not a lexical one. The right answer might be phrased completely differently from the question itself. A string match would miss valid grounded answers and pass invalid ones just as often.

Q. What's the first thing you'd add to this project before calling it production-ready?

A. Metadata on ingestion, document version and effective date at minimum, so retrieval can filter to current policy only, plus real embeddings instead of the placeholder swap noted in the code.

SCENARIO PREP

Scenario: Asked why your RAG project sometimes retries even on seemingly simple questions.

Approach: Check chunk size and overlap first. Nine times out of ten that's the real answer, not a deeper model or prompt issue.

Project C — Resumable LangGraph Agent

Objective: the same tool-calling agent as Project A, rebuilt on LangGraph so it can pause for approval and resume after a restart. Full graph code is on page 22 (AgentState, call_model, should_continue, checkpoint). This project wraps that graph in a runnable CLI:

```
# requirements.txt (adds to Project A's)
langgraph==0.3.* langgraph-checkpoint-sqlite==2.*

# run.py
from langgraph.checkpoint.sqlite import SqliteSaver
graph = builder.compile(checkpointer=SqliteSaver.from_conn_string("checkpoints.db"), interrupt_before=["tools"])
config = {"configurable": {"thread_id": "cli-session-1"}}

state = graph.invoke({"messages": [{"user": "Transfer $1500 from ACCT-10293"}]}, config)
pending = graph.get_state(config)
print("Pending tool call:", pending.values["messages"][-1].tool_calls)
approve = input("Approve? (y/n): ")
if approve == "y":
    graph.invoke(None, config) # resumes exactly where it paused
```

Expected output: the run stops before executing the transfer tool, prints the pending call for review, and only continues on explicit approval.

Troubleshooting: if resume re-runs from the start instead of continuing, confirm the same thread_id is passed on both calls — that's the checkpoint key.

PROJECTS D & E — ARCHITECTURE SUMMARY

PROJECT D: PLANNER → EXECUTOR → CRITIC

Three LangGraph nodes in sequence with a conditional loop back from Critic to Planner when the critic rejects the result. Reuses the AgentState shape from Project C, adding a critique field. Good next build once Projects A–C are working.

PROJECT E: MULTI-AGENT SYSTEM

Supervisor graph (page 22) wired to three specialist sub-graphs (fraud, policy, risk), each independently testable before being composed. Build and test each specialist in isolation first — debugging a 4-agent graph from scratch is significantly harder than composing three proven pieces.

Interview tip: Being able to say "I built the resumable version with an interrupt before the risky tool call" is a much stronger interview answer than describing human-in-the-loop in the abstract — it shows you've actually hit the checkpoint/resume mechanics, not just read about them.

HOW THESE FIVE PROJECTS BUILD ON EACH OTHER

A is the floor: one model, two tools, a visible trace. B adds the retrieval-with-self-correction loop on top of the same agent shape. C takes A's exact logic and rebuilds it as a resumable graph — same behavior, different runtime guarantees. D and E are compositional: D chains three of C's graph nodes into a quality loop, and E composes multiple complete C-style graphs under one supervisor. If you can only build one before an interview, build C — it's the one that demonstrates production-readiness rather than just correctness.

SCENARIO PREP

Scenario: Asked to prove you understand resumability, not just describe it.

Approach: Offer to trace what happens if the process crashes mid-approval. The thread_id and checkpoint mechanics are the concrete proof; the concept alone isn't.

Capstone 1 & 2 — Meta-Inspired & Apple-Inspired

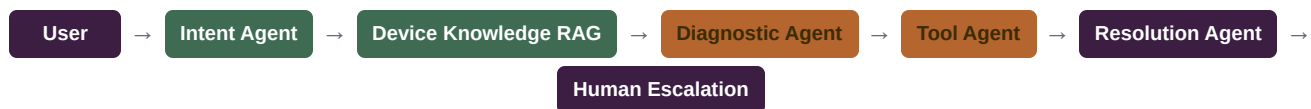
Framing: these are interview-style capstone designs built around each company's public product domain — not real internal architectures, and not claimed to be what these companies actually run.

1. META-INSPIRED: CONTENT INTELLIGENCE & MODERATION AGENT



Why agentic: moderation decisions require weighing policy text, context, and prior violations together — a fixed rules engine can't hold that much nuance, but full autonomy on takedown decisions is also too risky, hence the escalation path for borderline cases. **Design decision:** the RAG layer over policy documents is separate from the recommendation data tools so a policy update doesn't require touching the recommendation logic at all.

2. APPLE-INSPIRED: ON-DEVICE SUPPORT ASSISTANT



Why agentic: diagnosing "my phone won't charge" needs a branching sequence of checks (cable? port? battery health? software?) that depends on each prior answer — a scripted decision tree gets unmanageable past a handful of branches. **Focus areas:** privacy (diagnostics run on-device where possible, only escalating anonymized signals to cloud tools), tool permissions scoped tightly so the Tool Agent can read device diagnostics but never user content, and a hybrid on-device/cloud split so simple checks never leave the device.

Interview tip: For both of these, the strongest answer names the human-escalation boundary unprompted — content moderation and device diagnostics both have real user-harm potential if the agent acts wrongly with full autonomy.

INTERVIEW Q&A

Q. For the Meta-inspired project, why keep Memory as a separate box instead of folding it into the Planner?

A. Because they serve different purposes and different retention rules. The planner's reasoning is transient to one request, while memory (prior violations, past appeals) needs its own retention and privacy policy, often with a much longer or user-specific lifecycle. Conflating them makes it much harder to apply a data-retention policy correctly later.

Q. For the Apple-inspired project, why does the Tool Agent only see diagnostics, never user content?

A. Least privilege applied at the architecture level, not just as a policy statement. If the Tool Agent's credentials genuinely can't reach user content, a prompt injection or reasoning error in that agent can't leak it, regardless of what the model is tricked into attempting.

SCENARIO PREP

Scenario: Asked to defend the Apple capstone's on-device split under a 'why not just use the cloud' challenge.

Approach: Lead with privacy and latency together. On-device keeps sensitive diagnostics local and cuts round-trip time for the checks that run most often anyway.

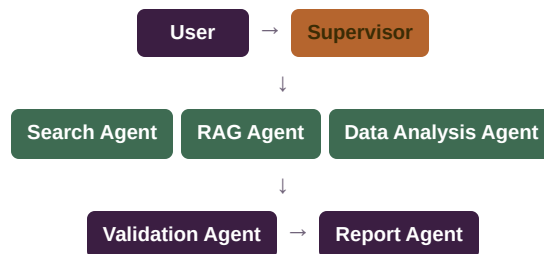
Capstone 3 & 4 — Tesla-Inspired & Google-Inspired

3. TESLA-INSPIRED: VEHICLE DIAGNOSTICS & SERVICE INTELLIGENCE AGENT



Why agentic: telemetry anomalies (a sensor reading trending outside normal range) need correlation against service history and manual procedures before a recommendation makes sense — a threshold alert alone produces too many false positives. **Design decision:** the Data Agent handles streaming/time-series ingestion completely separately from reasoning, so the Diagnostic Agent works off pre-aggregated windows, not raw telemetry — keeps the reasoning agent's context small and cheap. **Safety control:** Risk Evaluation gates any recommendation implying an immediate safety concern straight to a service alert, bypassing the conversational recommendation step entirely.

4. GOOGLE-INSPIRED: ENTERPRISE RESEARCH & KNOWLEDGE AGENT



Why agentic: a research brief ("summarize competitor pricing moves this quarter") needs web search, internal document retrieval, and quantitative analysis combined — three genuinely different tool sets, which is exactly the separable-sub-domain test from page 26 for justifying multi-agent. **Design decision:** Validation Agent runs independently of the three data-gathering agents specifically so it isn't biased by having produced the findings itself — the same principle as the Critic pattern on page 16.

INTERVIEW Q&A

Q. For the Tesla-inspired project, why does the Data Agent stay separate from the Diagnostic Agent instead of one agent handling both?

A. Streaming telemetry ingestion is a high-volume, low-reasoning problem. You want it running continuously and cheaply, likely without an LLM in the hot path at all. The Diagnostic Agent only needs to reason over pre-aggregated windows, not raw signal data, so merging them would force an expensive reasoning model to process volume it doesn't need to see.

Q. For the Google-inspired project, what would you monitor to catch the Validation Agent silently rubber-stamping findings?

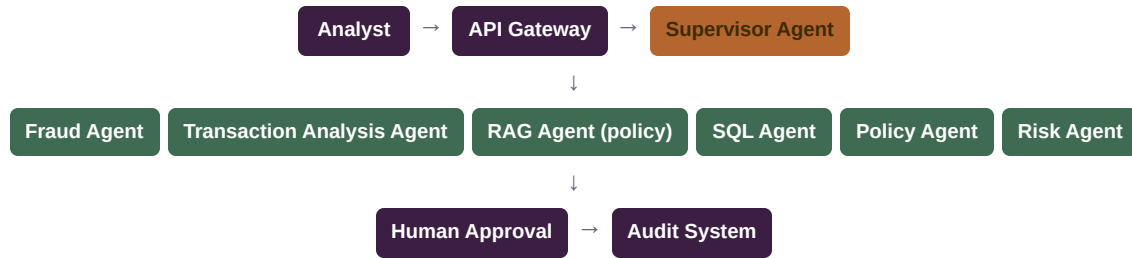
A. Track the Validation Agent's reject/flag rate over time. If it's approving essentially everything the other agents produce, that's a signal it's not actually adding independent scrutiny, and I'd audit a sample of approved reports by hand to confirm.

SCENARIO PREP

Scenario: Asked what you'd cut first from the Google-inspired capstone with half the agent budget.

Approach: Cut Search or Data Analysis before you touch Validation. An unvalidated report does more damage than a narrower one ever would.

Capstone 5 — AI Banking Operations & Fraud Investigation Agent



This is genuinely realistic of patterns common in financial-services engineering — not a claim about any specific bank's internals. An analyst flags a suspicious transaction; the supervisor delegates to the Fraud Agent (pattern analysis against known fraud signatures), the SQL Agent (pulls transaction/account history), the RAG Agent (retrieves the relevant compliance policy for this transaction type), and the Risk Agent (scores severity). Findings converge back at the supervisor, which drafts a recommendation — but never executes an action itself.

Why full autonomy is withheld here: freezing an account or reversing a transaction is expensive to undo and carries real regulatory and customer-harm consequences if wrong. The agent's job is to investigate fast and build a well-evidenced case; a human makes the final call and that decision is logged to the audit system with the full evidence trail attached. This isn't a technology limitation — it's a deliberate control, and naming it unprompted is a strong signal in an interview.

INTERVIEW Q&A

Q. What happens if the Fraud Agent and Policy Agent disagree on whether a transaction is suspicious?

A. The disagreement itself becomes part of the case file rather than being silently resolved by whichever agent ran last. The supervisor surfaces both findings to the human reviewer, with each agent's reasoning attached, since a disagreement between a pattern-based signal and a policy-based signal is often exactly the case that most needs human judgment.

Q. How would you evaluate this system before it ever touches a real case?

A. A golden dataset of past investigated cases with known outcomes, measuring whether the Fraud and Risk agents' flags match the historical determination, plus whether the RAG Agent cites the actual policy clause that applied, not just a plausible-sounding one. I'd also track false-positive rate carefully, since an agent that over-flags erodes analyst trust just as fast as one that misses real fraud.

KEY DESIGN POINTS FOR THIS CAPSTONE

DATA FLOW

Analyst request → supervisor delegates in parallel to Fraud/SQL/RAG/Risk → findings converge → draft recommendation → human approval → audit log, regardless of outcome.

SECURITY & COMPLIANCE

Every agent's DB credential is read-only; only the (human-gated) execution step can write. Full audit trail satisfies regulatory review requirements independent of the AI system itself.

SCENARIO PREP

Scenario: Asked directly: 'why not let the agent just act — humans are slow.'

Approach: Reframe speed as investigation speed rather than action speed. The agent already compresses hours of analyst work into minutes; the human gate only covers the one step where being fast and wrong actually costs something.

Explaining Any Capstone at Any Depth

Use this structure for all five projects — interviewers often ask for a specific time budget, so practice compressing and expanding the same material.

LENGTH	WHAT TO INCLUDE
2 minutes	The problem, the one-sentence architecture (supervisor + N specialists, or single agent + RAG), and why agentic beats the deterministic alternative.
5 minutes	Add: the specific agents/tools and what each owns, the one design decision you're proudest of, and the human-checkpoint or safety control.
10 minutes	Add: failure modes and how you'd monitor for them, how you'd evaluate it, how you'd scale it, and what you'd cut first if given half the budget.

THE QUESTION SET THAT APPLIES TO EVERY CAPSTONE

- Why Agentic AI here, specifically — what makes this not a fixed workflow?
- Why not simple RAG instead of a full agent?
- Why this memory architecture, or why none?
- Why this vector database / retrieval strategy?
- How would you scale it to 10x traffic?
- How would you secure it against the threats on page 28?
- How would you monitor and evaluate it in production?
- What happens if the LLM call fails mid-task?
- What happens if a tool fails, or RAG retrieves wrong information?
- How do you implement the human-approval step, concretely?

Interview tip: Answer "what happens if X fails" with the actual mechanism (circuit breaker, retry, fallback to cached answer, escalate to human) rather than "we'd handle it gracefully." Specificity here is what separates a senior answer from a junior one.

DESIGN-DECISION REASONING I'D ACTUALLY GIVE

For the banking capstone specifically: "I split Fraud and Policy into separate agents because they optimize for different things — recall for fraud, precision for policy citations — and merging them made tool selection worse in testing. I kept execution entirely out of agent hands because the cost of a wrong autonomous freeze is asymmetric with the cost of a 10-minute human review delay." That's the level of specificity to aim for on every design choice you present.

SCENARIO PREP

Scenario: Given only 2 minutes but asked for the 10-minute version anyway.

Approach: Say explicitly what you're cutting, 'I'll skip failure modes given the time,' rather than rushing all five layers. Naming the cut is its own signal of judgment.

Beginner & Intermediate Tiers

P0 Beginner

Q. What is an AI agent?

A. A system where an LLM decides which action to take next, in a loop, rather than just generating one response.

Q. What's the difference between a chatbot and an agent?

A. A chatbot generates replies; an agent decides and executes actions via tools.

Q. What is tool calling?

A. The model outputs a structured request to invoke a function, which the runtime executes and returns as an observation.

Q. What is RAG?

A. Retrieval-Augmented Generation: grounding a model's answer in retrieved documents rather than parametric memory alone.

Q. What's the difference between RAG and Agentic RAG?

A. RAG always retrieves then generates; Agentic RAG decides whether/what/when to retrieve and can re-query if the first pass is weak.

Q. What is LangChain used for?

A. Standardized interfaces for models, tools, prompts, and retrievers so an agent can be assembled from common building blocks.

Q. What is LangGraph and why not just use LangChain?

A. Graph-based orchestration with explicit state and checkpoints, needed once an agent requires pausing, resuming, or cycles.

Q. What is a hallucination, in agent terms?

A. A confident but unsupported claim not grounded in retrieved context or tool output.

P1 Intermediate

Q. Compare ReAct and Plan-and-Execute.

A. ReAct reasons and acts one step at a time visibly; Plan-and-Execute commits to a full plan up front and only replans on failure.

Q. Why would you cap an agent's max iterations?

A. To prevent infinite or runaway loops from burning cost/time when the model can't converge on an answer.

Q. What's the risk of giving an agent too many tools?

A. Tool selection accuracy drops measurably past roughly 15–20 tools; the model starts picking plausible wrong ones.

Q. What's the difference between short-term and long-term memory?

A. Short-term is scoped to the current run; long-term persists across sessions, usually in a database or vector store.

Q. What's a Supervisor agent's role?

A. Delegates sub-goals to specialist agents and synthesizes their results; doesn't do the specialist work itself.

Q. What is groundedness, and how do you check it?

A. Whether an answer is actually supported by retrieved context; typically checked via an LLM-as-judge score before returning the answer.

Q. Why does reranking improve RAG quality?

A. Initial vector search optimizes for recall; a reranker re-scores the top candidates specifically for relevance, improving precision.

Q. What's the difference between a workflow and an agentic workflow?

A. A workflow is fully fixed; an agentic workflow has fixed stages with an agent making decisions within each stage.

Q. Why use Pydantic for tool arguments?

A. It validates the model's structured output against a schema before executing the tool, catching malformed calls early.

SCENARIO PREP

Scenario: Blanking on a beginner-level question under pressure.

Approach: Restate the term in your own words, even imperfectly. 'Let me think out loud' beats silence, and recovery counts for more than the stumble did.

Advanced & Architect Tiers

P1 Advanced

Q. How do you prevent memory poisoning in a long-lived agent?

A. Validate tool output before it's written to long-term memory, scope what's allowed into long-term storage at all, and version writes so bad entries can be rolled back.

Q. How do you defend against indirect prompt injection from retrieved documents?

A. Treat retrieved content as data, not instructions, in the prompt structure; keep tool permissions narrow so even a successful injection has limited blast radius.

Q. How would you reduce an agent's cost by 50% without hurting quality?

A. Instrument cost per layer first, then route simple/classification steps to a small model, cache repeated queries, and cap retrieval to fewer, reranked chunks.

Q. When does multi-agent add latency without adding value?

A. When the sub-tasks aren't genuinely independent or specialized. Coordination overhead between agents can then exceed what one well-scoped agent would take.

Q. How do you evaluate an agent beyond just the final answer?

A. Track tool selection accuracy, planning accuracy, retrieval precision, step count, and groundedness alongside task success, not final-answer correctness alone.

Q. Why might LLM-as-judge give misleading evaluation scores?

A. It can share blind spots with the model it's judging and is sensitive to length/phrasing; mitigate with a stronger judge model and periodic human-labeled validation.

Q. What's the difference between Generator-Critic and Evaluator-Optimizer patterns?

A. Critic gives qualitative feedback for revision; Optimizer gives a quantitative score the loop checks against a numeric threshold.

Q. How do you handle a tool schema mismatch from the model?

A. Return a structured validation error back into the agent's context so it can retry with corrected arguments, rather than failing the whole run.

P2 Architect-Level

Q. How would you decide between a single powerful agent and a multi-agent system for a new project?

A. Check for genuinely separable sub-domains needing different tools/models; if one focused agent handles it reliably, that's the lower-risk, lower-cost choice.

Q. Design a model-routing strategy for cost control across a large agent fleet.

A. Classify request complexity cheaply first, route simple/classification work to a small model and reasoning-heavy work to a frontier model, with routing decisions themselves logged for tuning.

Q. What's your framework for deciding an action needs human approval?

A. Irreversibility and blast radius. If a wrong autonomous action is costly or hard to undo, it gets a checkpoint regardless of how confident the model is.

SCENARIO PREP

Scenario: Asked an architect-level question you've genuinely never considered.

Approach: Apply the recurring framework live: can this be deterministic, what's the blast radius. Doing that reasoning out loud, in real time, is itself the answer being tested.

Scenario-Based Interview Questions

SCENARIO	DIAGNOSIS & FIX
Agent keeps calling the same tool repeatedly	Check whether the tool result is actually merged back into state the model sees — usually a state-wiring bug, not a prompt issue. Fix the merge; add a hard step cap as a backstop regardless.
Agent hallucinates numbers in a financial summary	No groundedness check before returning. Add an LLM-as-judge or rule-based check that every number traces to a retrieved/tool-sourced value; reject and regenerate if not.
RAG retrieves outdated policy versions	Missing metadata filtering by document version/date, or stale index. Add an "effective date" filter and a re-indexing job triggered on document updates, not a fixed schedule.
Two agents in a multi-agent system produce conflicting answers	No arbitration rule. Add an explicit conflict-resolution step at the supervisor — surface both findings with reasoning rather than silently picking one.
Latency spikes under load	Check whether independent steps are running sequentially. Parallelize what can run concurrently; add caching for repeated queries; consider routing simple steps to a faster model.
Cost has crept up 3x since launch with no feature change	Likely unbounded context growth (unsummarized history) or a cache that stopped hitting. Instrument tokens per request over time to find where the growth started.
An attacker's document caused the agent to leak internal data	Indirect prompt injection via retrieved content. Isolate retrieved text as data (not instructions) in the prompt, and tighten the output tool's permission scope so even a successful injection can't exfiltrate broadly.
A tool integration that worked in staging fails intermittently in production	Missing timeout/retry handling for a flaky downstream API under real load. Add explicit timeouts, exponential backoff, and a circuit breaker; alert on the breaker tripping rather than on every individual failure.
An agent's memory keeps referencing a customer's old address after they updated it	Long-term memory wasn't invalidated on the update event. Write memory updates transactionally with the source-of-truth update, or better, don't cache facts that have an authoritative system of record — fetch them live instead.
A supervisor keeps re-delegating the same sub-task to a specialist that already completed it	Shared state isn't being checked before delegation. Have the supervisor read completed-task state before assigning, not just after receiving a result.
Data quality issue: the SQL Agent returns technically correct but misleading results	A query that's syntactically valid but semantically wrong for the question (e.g., missing a status filter, so cancelled transactions inflate a total). Add a semantic validation step comparing the query's intent to its actual WHERE clause before trusting the result.

Interview tip: For every scenario, name the monitoring signal you'd add so the same failure surfaces automatically next time — not just the one-time fix. That closing-the-loop habit is what interviewers are listening for at the senior level.

SCENARIO PREP

Scenario: Given a scenario with two equally plausible root causes.

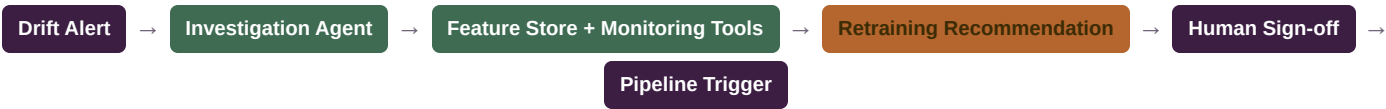
Approach: Say both, and explain how you'd tell them apart using the trace. A confident single guess is weaker than showing you'd verify before touching anything.

ML Engineering + Agentic AI

Agents increasingly sit alongside traditional ML pipelines rather than replacing them — the agent handles reasoning and orchestration, the ML pipeline still owns training and serving.

ML COMPONENT	HOW AN AGENT INTERACTS WITH IT
Feature stores	A tool the agent calls to fetch current feature values for a prediction, instead of the agent guessing or recomputing.
Model monitoring	An agent can be the responder to a drift alert — investigating which features shifted and drafting a retraining recommendation.
Model deployment	A deployment agent can run pre-flight checks (data schema match, latency benchmark) before promoting a model, with human sign-off for production.
Model evaluation	An agent can orchestrate running a candidate model against a golden set and summarizing the diff from the current production model.
Experiment tracking	A research agent can query MLflow/experiment logs to compare runs and recommend which configuration to promote.

ML ENGINEER AGENT — ARCHITECTURE SKETCH



INTERVIEW Q&A

- Q. Would you let an agent trigger a model retraining pipeline autonomously?**

A. I'd let it recommend and stage the retraining job, not trigger it unattended. Retraining consumes real compute budget, and a bad drift diagnosis could kick off unnecessary or even harmful retraining cycles. The gate is cheap; the mistake it prevents isn't.
- Q. How is a "model monitoring agent" different from a standard alerting rule?**

A. A rule fires on a threshold; an agent additionally investigates, correlating which features drifted, checking recent data pipeline changes, and producing a diagnosis instead of just a notification.
- Q. Where does this pattern break down — when would you NOT want an agent involved in an ML pipeline at all?**

A. High-frequency, low-latency inference paths (fraud scoring on every transaction, for instance) should stay pure ML serving. No LLM belongs in that hot path. The agent belongs in the surrounding operational loop (monitoring, investigation, retraining decisions), not in the prediction path itself.

REAL-WORLD EXAMPLE

A model monitoring agent I'd design for a credit-scoring model watches for population stability index drift on key features. When it fires, the agent pulls the last 30 days of scored applications, correlates the drift against a recent upstream schema change in the feature store, and drafts a summary for the ML team — often cutting a multi-hour manual investigation down to a five-minute review.

SCENARIO PREP

Scenario: Asked whether you'd let an agent auto-remediate a broken ML pipeline.

Approach: Separate diagnosis from remediation explicitly. Auto-diagnosing is low-risk; auto-remediating a live pipeline usually isn't.

The Solution Architect Lens & When NOT to Use Agentic AI

HOW AN LLM SOLUTION ARCHITECT APPROACHES A PROJECT

- **1. Business requirement** — what decision or action is actually needed?
- **2. Agent suitability** — does this need dynamic reasoning at all?
- **3. Architecture tier** — lowest tier (page 8–9) that solves it
- **4. Model selection** — smallest model that meets the quality bar
- **5. Tool & RAG decisions** — only what's demonstrably needed
- **6. Memory decision** — justified by a real personalization/continuity need
- **7. Multi-agent decision** — only for genuinely separable sub-domains
- **8. Security** — threat model from page 28 applied explicitly
- **9. Cost** — projected per-request cost before, not after, launch
- **10. Evaluation** — golden dataset defined before build starts
- **11. Deployment & monitoring** — tracing in place before go-live, not added after an incident
- **12. Governance** — who owns the tool registry, who approves new permissions

The recurring question at every one of these steps: "Can this be deterministic? Do we really need an agent here?" Senior architects ask this before, not after, choosing agentic AI.

28. WHEN NOT TO USE AGENTIC AI

SIMPLE, DETERMINISTIC LOGIC

CRUD operations, fee calculations, fixed-order onboarding steps — anything fully enumerable as rules.

SAFETY-CRITICAL AUTONOMOUS ACTION

Anywhere a wrong autonomous action causes irreversible harm and there's no time for a human checkpoint.

EXTREMELY LOW-LATENCY PATHS

Sub-100ms requirements can't absorb even one LLM round trip, let alone a multi-step loop.

COST-SENSITIVE, HIGH-VOLUME TASKS

Millions of simple, uniform requests where a classifier or rule is orders of magnitude cheaper.

Do not use an agent just because you can. The strongest interview signal isn't knowing how to build an agent — it's knowing when a workflow, a classifier, or a plain function is the better engineering decision.

INTERVIEW Q&A

Q. A stakeholder insists on an agent for a use case that's clearly simple CRUD. How do you handle that conversation?

A. I'd ask what problem they're actually trying to solve rather than pushing back on the solution directly. Often "agent" is just shorthand for "smarter" or "more flexible," and once we name the real requirement, a simpler implementation usually satisfies it at a fraction of the cost and with far more predictable behavior. If after that they still want agentic flexibility for a genuine future-proofing reason, that's a legitimate trade-off to make explicitly, not by default.

SCENARIO PREP

Scenario: Asked to review someone else's agent design and find the flaw.

Approach: Walk the 12-step architect framework on this page as a checklist. It's faster and more complete than a free-form critique would be.

Cheat Sheets (1/2)

AGENTIC AI LLM decides the next action in a loop — not just generates text. Needs: goal, plan, tool use, observation, stop condition.	ARCHITECTURE (6 TIERS) Simple → Tool-using → +RAG → +Memory → Production → Multi-agent. Pick the lowest tier that works.
AGENT LOOP Goal → Understand → Plan → Select Tool → Execute → Observe → Evaluate → Re-plan → Final Answer.	REACT Thought → Action → Observation, repeated. Visible reasoning; one LLM round trip per step.
LANGCHAIN @tool defines a callable; create_tool_calling_agent + AgentExecutor runs the loop; bind_tools for manual control.	LANGGRAPH StateGraph + typed state + add_conditional_edges + checkpointer for pause/resume; cycles = loop-back edges.

QUICK DECISION CHEATS

IF YOU NEED...	REACH FOR...
Fixed, predictable steps	A workflow, not an agent
Dynamic tool decisions, no resume needed	LangChain AgentExecutor
Pausable, resumable, human-approved steps	LangGraph with checkpointer + interrupts
Retrieval that's sometimes unnecessary	Agentic RAG, not fixed RAG
Several genuinely separable sub-domains	Supervisor + specialist multi-agent
An irreversible or high-value action	Human-in-the-loop checkpoint, never full autonomy

ACRONYM GLOSSARY

RAG Retrieval-Augmented Generation	REACT Reason + Act (interleaved)	HITL Human-in-the-loop
RBAC Role-Based Access Control	SSRF Server-Side Request Forgery	P50/P95 Median / 95th-percentile latency

SCENARIO PREP
Scenario: You freeze during a live coding round on LangGraph syntax.
Approach: Say the shape out loud first: state, nodes, conditional edges, checkpointer. Fill in the syntax after; structure recall tends to unlock syntax recall.

Cheat Sheets (2/2)

AGENTIC RAG

Decide-if-retrieve → retrieve → check groundedness → re-query if weak → synthesize. Retrieval is an action, not a fixed step.

MEMORY

Short-term = this run. Long-term = across sessions. Risks: poisoning, staleness, leakage, context explosion.

SECURITY

Treat retrieved/tool content as data, not instructions. Least privilege per tool. Sandbox execution. Audit-log every call.

TROUBLESHOOTING

Always start from the trace. Isolate the failing layer (plan / tool / retrieval / synthesis) before changing anything.

MULTI-AGENT

Supervisor delegates, specialists report back to supervisor only (never peer-to-peer) — prevents deadlock.

TOOL CALLING

Schema-validate every call (Pydantic). Cap tool count (~15–20 max). Two independent limits for risky actions: schema cap + code-level threshold.

EVALUATION

Evaluate tool accuracy, planning, retrieval, groundedness, cost, steps — not just final-answer correctness.

PRODUCTION

Gateway → Orchestrator → Runtime → Tools/RAG → Memory/Cache → Observability. Circuit breakers + timeouts on every external call.

SYSTEM DESIGN ONE-LINER

When asked to design any agentic system: state the goal, sketch the eight-layer stack (page 7), name the lowest architecture tier that fits, call out the security and human-checkpoint boundary, then discuss scaling and cost last — in that order.

THE EIGHT-LAYER STACK, ONE MORE TIME



If you remember nothing else from this guide: ask "can this be deterministic?" before reaching for an agent, keep humans in the loop on anything irreversible, and always be able to point to the trace when something goes wrong. Everything else in these 53 pages is detail in service of those three habits.

SCENARIO PREP

Scenario: Asked to pick one pattern from this page to defend as 'the most important.'

Approach: There isn't one. The strongest answer names whichever pattern matters for the specific system just discussed instead of naming a universal favorite.

How to Explain It in an Interview

30 SECONDS — "WHAT IS AGENTIC AI?"

"It's an LLM-driven system where the model itself decides what happens next — not just generating a reply, but planning steps, calling tools, checking the results, and adjusting the plan if something's off. The difference from a regular chatbot is that decision loop."

60 SECONDS — "HOW DOES AN AI AGENT WORK?"

"It runs a loop: understand the goal, plan a sequence of steps, pick a tool for the current step, execute it, look at what came back, and decide whether that's enough to answer or whether it needs to try something else. Frameworks like LangGraph make that loop explicit as a graph with state, instead of a hidden while-loop, so you can pause it, resume it, or drop in a human-approval step. The part that actually makes it 'agentic' rather than just automated is that the model is choosing the next action based on what it just observed, not following a fixed script."

2 MINUTES — "EXPLAIN AGENTIC AI ARCHITECTURE"

"I think of it as eight layers: the request comes through an API gateway into an orchestrator, which calls the LLM and a planner to figure out what to do. From there it can hit memory, tools, a RAG pipeline backed by a vector database, or external APIs and databases, depending on what the task needs. Once it executes an action, the result flows back through observation and evaluation, and if that's not sufficient the plan gets revised and it loops again; otherwise it returns the final response. Not every project needs all eight layers — a simple internal tool might just need the orchestrator, the LLM, and one or two tools. I always start from what the task actually requires and add layers only when there's a real reason."

5 MINUTES — "EXPLAIN A PRODUCTION AGENTIC RAG ARCHITECTURE"

"Traditional RAG always retrieves, then generates — Agentic RAG treats retrieval as a decision. So the flow is: understand the question, decide whether retrieval is even needed, pick which source to query if there are multiple, retrieve, and then check groundedness before committing to an answer — if the retrieved context doesn't actually support the question, rewrite the query and try again, up to a retry cap. In production I'd add hybrid search combining keyword and vector similarity, metadata filtering so you're only searching current, permission-appropriate documents, and a reranker on the top candidates before they reach the generator. On the infrastructure side, that whole path sits behind a gateway with caching for repeated queries, and every retrieval and generation step gets traced so a bad answer can be debugged back to the specific stage that failed, instead of treating the whole pipeline as a black box."

10 MINUTES — "DESIGN A MULTI-AGENT ENTERPRISE SYSTEM"

"I'd start by questioning whether multi-agent is even justified — only reach for it if there are genuinely separable sub-domains needing different tools or models. Assuming yes, I'd use a supervisor pattern: the supervisor decomposes the goal and delegates to specialists, who report results back to the supervisor only, never to each other, which avoids the deadlock and duplicate-work failure modes you get from peer-to-peer coordination. Each specialist gets its own scoped toolset under least privilege. For reliability I'd add circuit breakers around each specialist call, checkpointing so a mid-task failure doesn't restart everything, and a conflict-resolution rule at the supervisor for when two specialists disagree. On cost, I'd be upfront that a 4-agent system can mean 6-10 LLM calls for one user request, so I'd route simple delegation and classification calls to a cheaper model and reserve the frontier model for synthesis. And critically, for anything touching an irreversible action, the supervisor stops at a human-approval checkpoint rather than executing autonomously — that's a design decision, not a limitation."

SCENARIO PREP

Scenario: Asked to compress the 5-minute answer into 2 minutes on the spot.

Approach: Cut the deep-dive caveats first and leave the architecture intact. The shape of the answer should survive being shortened.

Top 50 Must-Remember Points

P0 ABSOLUTELY MUST KNOW (1–20)

- Agentic = the model decides the next action, not just generates text
- Agent = LLM + tools + a loop; Agentic AI = planning + self-correction on top
- RAG is a capability an agent can use, not a synonym for agent
- Pick the lowest architecture tier that solves the problem
- ReAct = visible Thought → Action → Observation per step
- Plan-and-Execute = plan once, execute, replan only on failure
- AgentExecutor's loop is implicit; LangGraph's is explicit and resumable
- LangGraph state, nodes, edges, conditional edges, checkpoints, cycles
- Agentic RAG treats retrieval as a decision, with a groundedness check
- Supervisor pattern: specialists report to supervisor only, never peer-to-peer
- Cap tool count (~15–20) — more tools measurably hurts selection accuracy
- Validate every tool call's arguments before executing (Pydantic schemas)
- Irreversible/high-value actions get a human checkpoint, never full autonomy
- Indirect prompt injection: malicious instructions hidden in retrieved content
- Treat retrieved/tool content as data, never as instructions
- Always start debugging from the trace, not from the prompt
- Evaluate tool accuracy, planning, retrieval, groundedness — not just final answer
- Cap max iterations on every agent loop, no exceptions
- Caching is usually the single biggest cost lever
- "Can this be deterministic?" is the question to ask before reaching for an agent

P1 IMPORTANT (21–38)

- Agentic workflow = fixed skeleton + agent deciding within each stage
- Multi-agent cost scales faster than agent count (coordination overhead)
- Memory: short-term (this run) vs. long-term (across sessions)
- Memory risks: poisoning, staleness, leakage, context explosion
- Hybrid search = keyword + vector; reranking improves precision after recall
- Metadata filtering restricts retrieval before similarity search runs
- Least privilege: every tool/agent gets minimum scope needed
- Sandboxing for any code-execution or file-access tool, always
- Circuit breakers stop retrying into a known downstream outage
- Model routing: cheap model for classification, frontier model for synthesis
- LLM-as-judge needs a stronger/different model than the one being judged
- Golden dataset regression before every agent change ships
- Idempotency keys prevent duplicate execution on retry
- Sequential vs. parallel: parallelize genuinely independent steps for latency
- Generator-Critic gives qualitative feedback; Evaluator-Optimizer gives a score
- Full observability trace: request ID through to final response, every hop
- Audit logging is mandatory wherever money, health, or legal outcomes are involved
- Batch APIs for non-interactive workloads cut cost significantly

P2 ADVANCED (39–50)

- Agent-as-a-Tool: hide a whole agent's loop behind a simple tool interface
- Tool-as-an-Agent: wrap a fragile deterministic tool with its own retry loop
- Event-driven agents trigger from queues/webhooks, not synchronous requests
- Map-Reduce agents: same agent applied to N chunks, then a reduce step
- Vector DB read throughput is usually the real scaling bottleneck at Tier 3+
- Hierarchical (supervisor-of-supervisors) only pays off past ~10 agents
- SSRF risk: URL-fetching tools can be tricked into hitting internal endpoints
- A wrong tool with right-looking arguments is often worse than no tool call

SCENARIO PREP

Scenario: Asked to name your weakest area on this list, unprompted.

Approach: Pick a real one, and pair it with how you'd close the gap in a week. Self-awareness reads better than claiming mastery of all 50 points.

7-Day Interview Revision Plan

DAY	TOPICS	PRACTICE	CHECKLIST BEFORE MOVING ON
1	Fundamentals, core characteristics (p.4–6)	Explain the spectrum table from memory; write the minimal agent loop by hand	Can state the one-line Agentic AI definition without notes
2	Architecture + design patterns (p.7–16)	Sketch the 8-layer stack and all 6 tiers from memory; pick a pattern and explain when NOT to use it	Can draw the architecture diagram unprompted
3	LangChain + LangGraph (p.17–22)	Run Project A end to end; convert it to a LangGraph version with a checkpoint	Project A runs locally and returns a grounded answer
4	Agentic RAG + Memory (p.21–25)	Run Project B; break it deliberately (bad chunk size) and observe the retry behavior	Can explain groundedness checking without looking it up
5	Multi-agent + Security (p.26–29)	Sketch a supervisor graph for a domain from your own work; list its threat surface	Can name 5+ threats from page 28 unprompted
6	Production + Troubleshooting (p.30–36)	Pick 5 troubleshooting entries and explain root cause + fix out loud, timed at 60 seconds each	Can debug a "stuck in loop" scenario end to end verbally
7	System design + mock interview (p.37–53)	Do a full mock: pick one capstone, deliver the 2-min then 10-min version; run through 15 interview-bank questions cold	Comfortable expanding/compressing the same architecture on demand

The night before: re-read the cheat sheets (p.49–50) and the P0 list on page 52 only. Don't cram new material — at this point, retrieval speed on what you already know matters more than adding anything new.

Closing note: the strongest interview answers in this material aren't the definitions — they're the "why not" answers: why not an agent here, why not full autonomy there, why not multi-agent for this. Practice those as hard as you practice the architecture diagrams.

SCENARIO PREP

Scenario: You only have 2 days before the interview, not 7.

Approach: Compress to Day 2 and 3 for architecture, and Day 7 for the mock interview. When time is short, fluency and live practice matter more than breadth.

Good luck. You've now covered the same material as a senior Agentic AI/LLM Solution Architect — the last thing that separates a good candidate from a great one in the room is saying the "why not" out loud before anyone asks for it. Go get the offer, Deepika.